

Database Warehousing Case Study

Critical evaluation and implmenetation of a Library database warehouse application

University of Gloucestershire

Word Count: 2659

CT6049 

18 May 2025

Table of Contents

1	The Dimensional Model	1
1.1	Overview	1
1.2	Decision Maker Questions	1
1.3	Star Schema	2
1.4	Other Schemas	2
1.4.1	Snowflake	2
1.4.2	Galaxy	2
1.5	Implementation	2
2	The ETL Process	3
2.1	Overview	3
2.2	Extract	4
2.3	Transform	4
2.4	Load	4
2.5	ETL vs ELT	4
3	Security, Performance, and Implementation	5
3.1	Authentication & User Accounts	5
3.2	Partitioning	6
3.3	Indexes	7
3.4	Prepared Statements	8
4	Backup and Restoration	8
	Bibliography	i
	Appendices	ii
A	Additional Information	ii
A.1	Application login details	ii
A.2	Database details	ii
A.3	Deployment Steps	ii
B	Additional Images	iii
B.1	Admin Panel	iv
C	Backend	iv
C.1	Main App	iv
C.2	Redis Setup Configuration	v
C.3	Controller - Authentication	vi
C.4	Controller - Backup	vii
C.5	Controller - Book	ix
C.6	Controller = ETL	xi
C.7	Controller - Loan	xii
C.8	Controller - Warehousing	xiv
C.9	DTO - Book	xvi
C.10	DTO - Loan	xvi
C.11	Model - Book	xvii
C.12	Model - Course	xviii
C.13	Model - Loan	xix
C.14	Model - Module	xxi
C.15	Model - Staff	xxii
C.16	Model - Student	xxiii
C.17	Model - User	xxiv
C.18	Repository - Book	xxv

C.19	Repository - Course	xxv
C.20	Repository - Loan	xxvi
C.21	Repository - Module	xxvi
C.22	Repository - Staff	xxvi
C.23	Repository - Student	xxvii
C.24	Repository - User	xxvii
C.25	Security - JWT Filter	xxviii
C.26	Security - JWT Utilities	xxix
C.27	Security - Configuration	xxxi
C.28	Service - Authentication	xxxiii
C.29	Service - Backup	xxxiv
C.30	Service - Book	xxxvi
C.31	Service - Cover	xxxvii
C.32	Service - ETL	xxxix
C.33	Service - Loan	xli
C.34	Service - Warehousing	xlili
D	Migration	xlvi
D.1	Operational Tables	xlvi
D.2	Warehousing Tables	xlviili
D.3	Operational Data Seeder	l
E	Frontend	lii
E.1	App Main	lii
E.2	API Scripts	liii
E.3	Data Stores	lv
E.4	Component - Alert	lvii
E.5	Component - BookCard	lvii
E.6	Component - Browse	lviii
E.7	Component - Login	lx
E.8	Component - Navbar	lxi
E.9	Component - Dashboard	lxi
E.10	Component - Staff Dashboard	lxii
E.11	Component - Student Dashboard	lxiv
E.12	Component - LoanPanel	lxvi
E.13	Component - Backup Panel	lxviii
E.14	Component - Chart Card	lxx
E.15	Component - Course Engagement Chart	lxxi
E.16	Component - Duration Chart	lxxii
E.17	Component - Fine Revenue Chart	lxxiv
E.18	Component - Loan Chart	lxxvi
E.19	Component - Overdue Chart	lxxviii
E.20	Component - Popular Chart	lxxx
E.21	Component - Year Engagement Chart	lxxxii
F	Miscellaneous	lxxxiii
F.1	Java Application Yaml	lxxxiii
F.2	Java pom.xml	lxxxiv
F.3	Nix Flake	lxxxvi
F.4	Docker Compose	lxxxviii

List of Figures

Figure 1	Star Schema Diagram	3
Figure 2	Example ETL query	4
Figure 3	AuthService Checking login details	5
Figure 4	Security Chain	6
Figure 5	Partitioning loans fact table	7
Figure 6	DB index definitions	7
Figure 7	getPeakDays Query Building	8
Figure 8	Debug Logging of Prepared Statement	8
Figure 9	Admin panel showing backups	9
Figure 10	Backup Service pg_dump	9
Figure 11	Backup Folder	10
Figure 12	Frontend restore backup handler	10
Figure 13	Admin panel showing ETL & Backup Buttons	iv

List of Tables

Table 1 Application account details	ii
Table 2 Postgresql Database details	ii

Chapter 1: The Dimensional Model

1.1 Overview

When talking about databases, people typically are referring to what's known as Operational databases. These are databases that “focus on managing real-time data”, perfect for applications like the one built for CT6049 A1 (Anna Litvinska, 2025). However, when it comes to gaining real historical insights into the flow of this data, using the same database is cumbersome and faces some significant issues.

Thus, the data warehouse type of database is often preferred. These databases are a “specialized system ..[which].. are optimized for complex analytical queries, supporting decision making processes.” (Anna Litvinska, 2025).

(Oracle, 2023) presents a short list of key benefits that are provided by deploying a data warehouse:

- **Subject-oriented** Data warehousing is more focused on the analyze “about a particular subject” rather than individual records. In the case of this library system, that would mean items like Loans, and Fines. Instead of inspecting each loan individually, the warehouse system allows users to monitor how loans change over time, or over course, etc.
- **Integrated** Unlike heavily normalized operational databases, warehousing focuses more on getting the information presented, in a fast, accurate manner. This also means pulling data from multiple different sources, all pooling together. The rules about normalisation and types are more relaxed.
- **Nonvolatile** These warehouses are meant to be kept running for year after year. Once data is part of a warehouse it rarely, if ever, changes. Even if the data was incorrect during operation time, the warehouse tracks all data and its inconsistencies.
- **Time-variant** Analysis of the data stored in a warehouse is typically more interested in the evolution of data over time; how fields were expanded, updated fields and schemas, etc.

1.2 Decision Maker Questions

As part of this assignment, we have been briefed to build a data warehousing system, that consumes the data produced by the previous library management application. To aid with guiding the development, the following Decision maker questions were produced. These questions try to simulate what people with decision making powers would want access to.

1. **How many total loans has their been over the past 3, 6, 12 months?**
2. **Which courses are engaging with the library system the most?**
3. **How many loans are / have been outstanding per course (fines that have still not been paid)?**
4. **Total revenue gained by fines collected per month**
5. **What days are books most often taken out?**
6. **What is the average loan duration by course? (are books being returned early)**
7. **Does year of study have an impact on library usage?**

1.3 Star Schema

Often times in a data warehouse application, users are only really interested in one core subject, and how that subject reacts to, and adapts over different events. This pattern leads to a “hub and spoke”, or star-like, way of thinking where the core subject table takes centre place with the events surrounding it.

This is standardly called the Star Schema, and refers to tables as either Dimensions (events) or Facts (the subject).

The key advantages of a star schema is simplicity; with a trade-off in normalisation, these schemas are able to become incredibly efficient at data retrieval, no matter the dimensions being observed. The core facts table holds foreign keys for each and every dimension surrounding it, this means queries that would be long chains of table joins are reduced down to a single join to the relevant event.

1.4 Other Schemas

As covered by Laiba Siddiqui (2026), Lindsay MacDonald (2025), and Amani Undru (2025) Star schema is not the only schema out there; Snowflake and Galaxy are also prominent choices; both with their upsides and downfalls.

1.4.1 Snowflake

Snowflake “uses the same basic concept” as Star, however it reapplies some of the normalisation that was lost. There is still a core fact table surrounded by dimension tables, however these dimensions are further reduced into sub-dimensions. This provides more of a structure and proper hierarchies, but reintroduces table joining, potentially reducing speed.

On the upside however, with normalised dimensions, values are only stored once, reducing duplication and redundancy resulting in a decrease to storage requirements.

1.4.2 Galaxy

Just as Snowflake built on top the foundations of Star, Galaxy inturn builds on top of Snowflake, expanding the complexity once again. However this time, providing a functionally different use case. For those business that have numerous fact tables, spread across many interconnected processes, Galaxy schema attempts to unify these whilst still keeping facts separated.

1.5 Implementation

For the development of the warehousing application, a standard star schema was converged upon. This provided the quickest query times possible, whilst keeping the database management simple. Figure 1 shows a diagram of the full warehousing system, containing just 1 fact table, and 4 dimension tables.

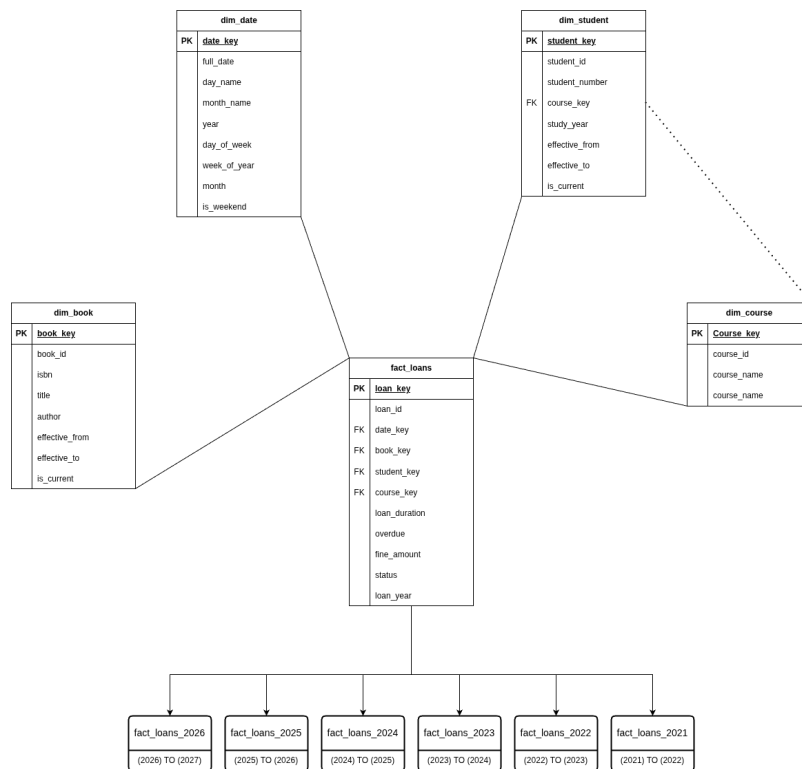


Figure 1: Star Schema Diagram

Chapter 2: The ETL Process

2.1 Overview

Having the design of the warehouse system nailed in, means nothing when there's no data actually being stored. That's where ETL comes in: Extract, Transform, and Load "is the process data-driven organizations use to gather data from multiple sources and then bring it together to support discovery, reporting, analysis, and decision making" (Oracle, 2021).

The ETL process is actually very simple, and follows a fully linear line of logic. Firstly data is **extracted** from the relevant sources, it is then **transformed** to match the structure set out by the warehouse tables. Then lastly this prepared data is **loaded** / copied into the corresponding location.

For the application developed for this assignment, the ETL process is handled by a dedicated ETL Service (Appendix C.32); currently only accessible through the corresponding controller (ap-controller-etl). This means all ETL processes are manually started, and is reliant on the administrators performing regular data refreshes. However, as the core ETL logic is exposed through a service, setting up a Spring Boot scheduled task would not require too much additional development time¹.

As a demonstration of ETL in implementation in DBA2 Figure 2 will be used as an example, with each stage being walked through.

¹<https://spring.io/guides/gs/scheduling-tasks>

```

1 private int loadDimStudent() {
2     String query = """
3         INSERT INTO warehouse.dim_student (student_id, student_number, course_key, study_year, effective_from, is_current)
4         SELECT student_id, student.student_number, dimc.course_key, student.study_year, CURRENT_DATE, true
5         FROM students student
6         JOIN warehouse.dim_course dimc ON dimc.course_id = student.course_id
7         WHERE student.student_id NOT IN (
8             SELECT student_id FROM warehouse.dim_student WHERE is_current = true
9         )
10     """;

```

Figure 2: Example ETL query

2.2 Extract

Starting with extraction, Listing 1 shows exactly all the code that belongs to this stage. Here two different “sources” of data are being used; the standard students table, and the dimc (dim_course) table. Counter productively the join that grants access to dim_course is technically part of transformation, not extracting.

```

SELECT student.student_id, student.student_number, dimc.course_key,
student.study_year, XXXXXXXXXXXX, XXXX
FROM students student

```

Listing 1: ETL - Extract

2.3 Transform

Listing 2 then shows that exact aforementioned join, along with the rest that fits into this stage. The transform stage is by-in-far the largest of the 3 sections, contributing a significant portion of the work. This is because any sort of filtering, creating values, or any modification of data in anyway is considered a transformation. This also means the two values censored in Listing 1 (CURRENT_DATE and true) are apart of this stage theoretically.

```

JOIN warehouse.dim_course dimc ON dimc.course_id = student.course_id
WHERE student.student_id NOT IN (
    SELECT student_id FROM warehouse.dim_student WHERE is_current = true
)

```

Listing 2: ETL - Transform

2.4 Load

Lastly is the load stage, here the transformed data is written into the new warehouse tables, the order of operations here is crucial. If tables are not populated in the right order, and schemas require referential integrity, then an out of time INSERT could cause the entire system to crash.

```

INSERT INTO warehouse.dim_student (student_id, student_number, course_key,
study_year, effective_from, is_current)

```

Listing 3: ETL - Load

2.5 ETL vs ELT

Whilst the above walked through the ETL process, the more apt name for what occurs is ELT. Featuring the exact same terms, Extract, Load, Transform, is a process in which “the transformation of data occurs after it is loaded” (Smallcombe, 2026). Due to the structure of the SQL queries being executed, the data being transformed never actually leaves the system, there is no third party, or temporary location in memory being used. In this small use case, there is no practical benefits to compare.

Chapter 3: Security, Performance, and Implementation

3.1 Authentication & User Accounts

DBA1 made for the first assignment of CT6049 implemented a stateless authentication system by making use of JWT (Json Web Token), and a simplistic unsecure user system; DBA2 has improved on both of these areas significantly.

Firstly, passwords are finally no longer stored in plain text. Through the use of BCrypt, user passwords are now hashed and salted securely server-side. Figure 3 shows the core linchpin behind all authentication in DBA2.

When a user submits their credentials to be checked, AuthService will try to retrieve the corresponding user through its username. If no records are found, the user is immediately denied access without wasting compute cycles of password checking. In the case that the user account has been found, a BCryptPasswordEncoder, hashes and verifies the passwords match. If all checks pass and the user is allowed to login, AuthService does one final database call, to see if the user logging in has a staff profile, associated with their ID.

If the user is actually a staff account, the Role embedded in the staffs table is retrieved, else a generic “STUDENT” role is applied. Both the users username and supplied role are passed into the generateToken method, which can be seen in Appendix C.26.

```
1 @Service
2 public class AuthService {
3     private final UserRepository userRepo;
4     private final StaffRepository staffRepo;
5     private final BCryptPasswordEncoder passwordEncoder;
6     private final JwtUtil jwtUtil;
7
8     public AuthService(UserRepository userRepo, StaffRepository staffRepo, BCryptPasswordEncoder passwordEncoder,
9         JwtUtil jwtUtil) {
10         this.userRepo = userRepo;
11         this.staffRepo = staffRepo;
12         this.passwordEncoder = passwordEncoder;
13         this.jwtUtil = jwtUtil;
14     }
15
16     public String attemptLogin(String username, String password) {
17         Optional<User> userOpt = userRepo.findByUsername(username);
18
19         if (userOpt.isEmpty())
20             return null;
21
22         User user = userOpt.get();
23
24         if (!passwordEncoder.matches(password, user.getPasswordHash()))
25             return null;
26
27         String role = staffRepo.findById(user.getId()).map(Staff::getRole).orElse("STUDENT");
28         return jwtUtil.generateToken(user.getUsername(), role);
29     }
30 }
```

Figure 3: AuthService Checking login details

These roles and subsequent JWT token are then used in 2 key places. On the frontend, the svelte application, gets the users role from decoding the tokens claims and uses the role attribute to conditionally display the correct components. Listing 4 is just one example of this conditional logic; using the main Dashboard.svelte component as a thin wrapper, Students are directed to their dashboard; whilst ADMIN and DECISION_MAKERS (which are sub-roles within Staff) are direct to their own component.

```

<script>
  import { userRole } from "../../lib/stores";
  import StaffDashboard from "./StaffDashboard.svelte";
  import StudentDashboard from "./StudentDashboard.svelte";
</script>

{#if $userRole === "STUDENT"}
  <StudentDashboard />
{:else if ["ADMIN", "DECISION_MAKER"].includes($userRole)}
  <StaffDashboard />
{:else}
  <p>Loading...</p>
{/if}

```

Listing 4: Dashboard Wrapper

The above “security” is more of a convince and isn’t actually trusted with keeping the application and the data stored secure. As with anything on the frontend, the components can be manually interfered with, therefore to bolster security a Spring SecurityFilterChain is employed; Figure 4.

All API endpoints are protected based on the users role, with varying degrees of privileges. If DBA2 implement more sub-roles, the granularity of security could be increased. As it is now, there are 4 access levers people fall into.

For the services required by everyone (even logged out users) `permitAll()` is applied. This covers things like the basic root top-level page, and also access to `/api/auth`. If a logged out user tried to manually navigate to one of the restricted endpoints, their token , or lack off, would deny them access and return them back to their previous location.

The first group with some actual access is the standard authorized students. These accounts can access all the same before but also the `/api/books/cover` endpoints allowing real usage of the library system.

Lastly, the three new endpoints in DBA2 are the `/api/etl/`, `/api/warehouse/`, and `/api/backup`. All three of which are locked down and require a staff account, with the endpoints that could result in data loss requiring an admin; in which there is only one account.

```

@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    var jwtFilter = new JwtAuthenticationFilter(JwtUtil);

    http.csrf(csrf -> csrf.disable())
        .cors(cors -> cors.configurationSource(request -> {
            var cfg = new CorsConfiguration();
            cfg.setAllowedOrigins(allowedOrigins);
            cfg.setAllowedMethods(List.of(a1: "GET", a2: "POST", a3: "PUT", a4: "DELETE", a5: "OPTIONS"));
            cfg.setAllowedHeaders(List.of(a1: ""));
            cfg.setAllowCredentials(true);
            return cfg;
        }));
    .authorizeHttpRequests(auth -> auth
        .requestMatchers("/api/auth/**", "/").permitAll()
        .requestMatchers(HttpMethod.GET, "/api/books/cover/**").permitAll()
        .requestMatchers("/api/warehouse/**").hasAnyRole("DECISION_MAKER", "ADMIN")
        .requestMatchers("/api/etl/**").hasRole("ADMIN")
        .requestMatchers("/api/backup/**").hasRole("ADMIN")
        .anyRequest().authenticated()
        .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class);
    // .httpBasic(Customizer.withDefaults());
    return http.build();
}

```

Figure 4: Security Chain

3.2 Partitioning

Database partitioning, referring to “breaking data in an applications database into separate pierces”, is a common technique used in data warehousing environments when tables get too unwieldy (Charlie Custer, 2023). Partitioning a table physical creates small sub-tables hat only hold a subset of

the whole range of records, and is practically useful in situations where you have macro scale groupings.

For example DBA2 employs partitioning, as shown by Figure 5, here the `fact_loans` table is being sub-divided out into smaller more manageable per-year tables. At the scale of data DBA2 is handling, not much benefit can be measured, however database partitioning can drastically reduce input and output times for more complex queries.

Any query filtering based on `loan_year`, in whole or in part, can now dramatically slice away entire swaths of records at once.

```
1 CREATE TABLE IF NOT EXISTS warehouse.fact_loans (  
2   loan_key INT GENERATED BY DEFAULT AS IDENTITY,  
3   loan_id VARCHAR(36) NOT NULL,  
4   date_key INT NOT NULL REFERENCES warehouse.dim_date (date_key),  
5   book_key INT NOT NULL REFERENCES warehouse.dim_book (book_key),  
6   student_key INT NOT NULL REFERENCES warehouse.dim_student (student_key),  
7   course_key INT REFERENCES warehouse.dim_course (course_key),  
8   loan_duration INT,  
9   overdue BOOLEAN NOT NULL DEFAULT FALSE,  
10  fine_amount DOUBLE PRECISION,  
11  status VARCHAR(20),  
12  loan_year INT NOT NULL,  
13  
14  CONSTRAINT fact_loans_id UNIQUE (loan_id, loan_year)  
15  ) PARTITION BY RANGE (loan_year) ;  
16  
17  
18 CREATE TABLE warehouse.fact_loans_2026 PARTITION OF warehouse.fact_loans FOR VALUES FROM (2026) TO (2027) ;  
19 CREATE TABLE warehouse.fact_loans_2025 PARTITION OF warehouse.fact_loans FOR VALUES FROM (2025) TO (2026) ;  
20 CREATE TABLE warehouse.fact_loans_2024 PARTITION OF warehouse.fact_loans FOR VALUES FROM (2024) TO (2025) ;  
21 CREATE TABLE warehouse.fact_loans_2023 PARTITION OF warehouse.fact_loans FOR VALUES FROM (2023) TO (2024) ;  
22 CREATE TABLE warehouse.fact_loans_2022 PARTITION OF warehouse.fact_loans FOR VALUES FROM (2022) TO (2023) ;  
23
```

Figure 5: Partitioning loans fact table

3.3 Indexes

Another technique often used in data warehousing databases is the idea of indexing. “An index is a special data structure used to speed up data retrieval” (itsimiro, 2025). Indexes are essentially markers, and ways of laying out tables in different ways that could prove faster than scanning each record individually.

Figure 6 shows all the indexes created for the warehouse application. Three separate tables received indexes, with the vast majority being applied to `fact_loan`. As a consequence of being the only fact table in the system, this table is under constant reads and joins with varying dimensions. For this reason each foreign key received its own index to drastically improve join speeds.

The other 2 index went to the `dim_book` table, purely because most queries contained something to do with fetching details about books. And lastly the date dimension table. This index was applied to help during the ETL process, as before the main dimensions are populated, a `dim_date` table is populated for referential stability.

```
24  
25 CREATE INDEX idx_fact_loans_date_key ON warehouse.fact_loans (date_key) ;  
26 CREATE INDEX idx_fact_loans_book_key ON warehouse.fact_loans (book_key) ;  
27 CREATE INDEX idx_fact_loans_student_key ON warehouse.fact_loans (student_key) ;  
28 CREATE INDEX idx_fact_loans_course_key ON warehouse.fact_loans (course_key) ;  
29 CREATE INDEX idx_Fact_loans_overdue ON warehouse.fact_loans (overdue) ;  
30  
31 CREATE INDEX idx_dim_date_full_date ON warehouse.dim_date (full_date) ;  
32 CREATE INDEX idx_dim_book_id ON warehouse.dim_book (book_id) ;
```

Figure 6: DB index definitions

Whilst indexes can drastically speed up the reading speeds of databases, they come with a lot of drawbacks aswell; as outlined by Augustine (2024). Once such drawback is the overhead that comes with storing, what are essentially separate caches of the same table. In terms of computation, using indices leads to increased memory usage, increased storage requirements, and most of note, drastically worsened write speeds.

That is however, less of a concern for warehouse databases, where writes are already few-and-far between, and always performed in bulk anyway.

3.4 Prepared Statements

According to (OWASP, 2025) Injection based attacks are still within the top 5 most common web vulnerabilities reported. Now while the application made for this assignment doesn't handle too much user input data, it's still good practice to use SQL prepared statements where possible.

Figure 7 shows the `getPeakDays` method used for the data collection for one of the dashboard graphs. The query, however, takes in user defined start and end values. Instead of manually concatenating the passed in values directly into the SQL string, named parameters were used.

```
public List<Map<String, Object>> getPeakDays(LocalDate start, LocalDate end) {  
    String query = ""  
        SELECT dimd.day_name, dimd.day_of_week, COUNT(*) AS loan_count  
        FROM warehouse.fact_loans factl  
        JOIN warehouse.dim_date dimd ON factl.date_key = dimd.date_key  
        WHERE (CAST(:start AS DATE) IS NULL OR dimd.full_date >= :start)  
            AND (CAST(:end AS DATE) IS NULL OR dimd.full_date <= :end)  
        GROUP BY dimd.day_name, dimd.day_of_week  
        ORDER BY dimd.day_of_week  
    "";  
  
    @SuppressWarnings("unchecked")  
    List<Object> results = entityManager.createNativeQuery(query)  
        .setParameter("start", start)  
        .setParameter("end", end)  
        .getResultList();  
}
```

Figure 7: `getPeakDays` Query Building

Figure 8 just provides extra validity to the above query being a complete prepared statement. By setting `logging.level.org.hibernate.sql: DEBUG` and `logging.level.org.hibernate.orm.jdbc.bind: TRACE` in the `application.yaml` file hibernate's debug messages are returned. Here the log shows the exact same query being sent with ? placeholders, demonstrating that this isn't just string interpolation. The postgresql driver is executing the binding.

```
SQL : SELECT dimd.day_name, dimd.day_of_week, COUNT(*) AS loan  
_count  
FROM warehouse.fact_loans factl  
JOIN warehouse.dim_date dimd ON factl.date_key = dimd.date_key  
WHERE (CAST(? AS DATE) IS NULL OR dimd.full_date >= ?)  
AND (CAST(? AS DATE) IS NULL OR dimd.full_date <= ?)  
GROUP BY dimd.day_name, dimd.day_of_week  
ORDER BY dimd.day_of_week
```

Figure 8: Debug Logging of Prepared Statement

Chapter 4: Backup and Restoration

Due to the expected operating time frame of warehousing databases, a suitable and robust backup strategy is practically essential. These systems could be the only records of years, of transactions, losing access to, or having the data corrupt would be an incalculable loss. Thankfully due to the choice of using Postgresql for DBA2, rudimentary backups are trivial.

As part of Postgresql standard suite of tools they provide `pg_dump`, a "utility for exporting". Whilst on the official documentation, it advises against using this as backups, for a database of this size and simplicity it's fine. (PostgreSQL, 2026)

DBA2 enrolls the use of this utility through a dedicated Backup Service, as seen fully in Appendix C.29, invoked exclusively through the Backup Controller (Appendix C.4). Admin accounts, as covered in Section 3.1, are the only accounts authorised to initiate, or restore, a backup. This is done entirely through the web dashboard, through the panel shown in Figure 9.

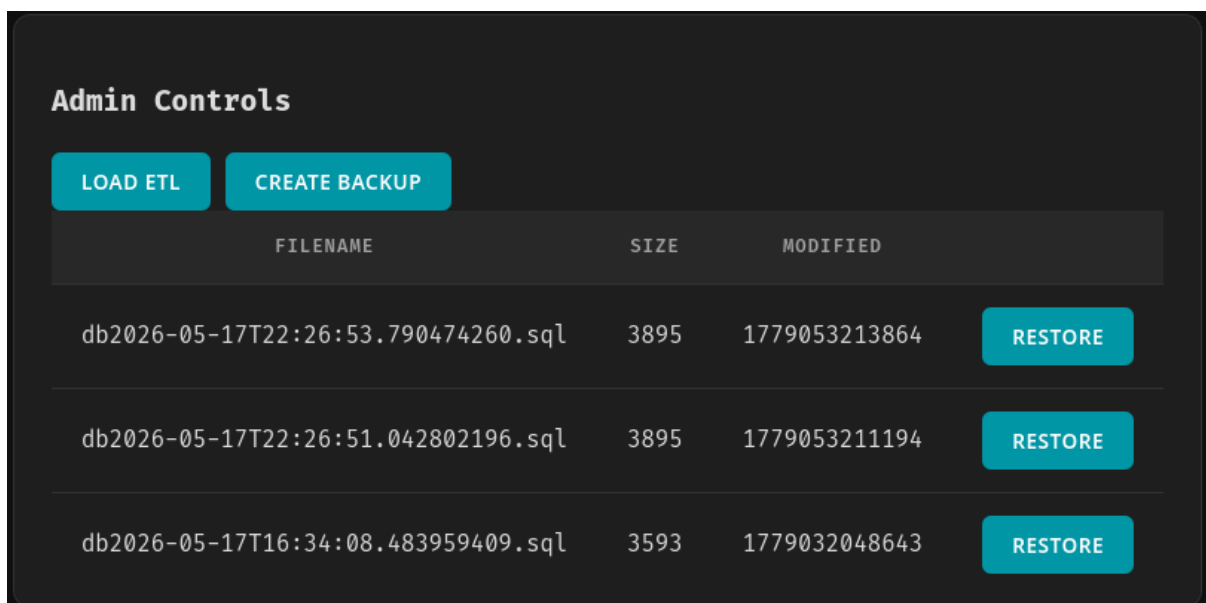


Figure 9: Admin panel showing backups

Figure 10 in turn, shows the backend service code that handles creating the actual backup. A `ProcessBuilder` is used to execute the `pg_dump` command as if it was a user in a terminal. Executing the command this way, means the running process has its own “shell environment”, allowing environment variables to be passed in (`process.environment()`...).

Whilst security has not been applied here for the demonstrations, using a process builder to construct the arguments will, in future, allow for complete configurability of the arguments, as well as proper secrets handling through `.env` files or `direnv`.

```
public String createBackup() {
    ensureBackupDir();
    String filename = "db" + LocalDateTime.now().toString() + ".sql";
    Path target = Paths.get(backupPath, filename);

    ProcessBuilder process = new ProcessBuilder(
        ...command: "pg_dump",
        "-h", "localhost",
        "-p", "5432",
        "-U", "libuser",
        "-d", "libdb",
        "-f", target.toString());

    process.environment().put(key: "PGPASSWORD", value: "libpass");
    process.redirectErrorStream(redirectErrorStream: true);

    try {
        Process proc = process.start();
        int exit = proc.waitFor();

        if (exit != 0) {
            String err = new String(proc.getInputStream().readAllBytes());
            throw new RuntimeException("pg_dump failed" + err);
        }
    }

    return filename;
}
```

Figure 10: Backup Service `pg_dump`

As for the actual backup files themselves, they are timestamped and dumped in a configurable but basic location, as shown by Figure 11. The table of available backups, on the admin panel, is populated from reading the files present in this folder; If a backup file is deleted, or moved, the ability to backup from that backup will be lost. This in turn however means additional backing up practices can be stacked ontop, as the backups are just general files. Volume snapshots could be taken, or mesh file syncing, or even git source control, could take over from this point.

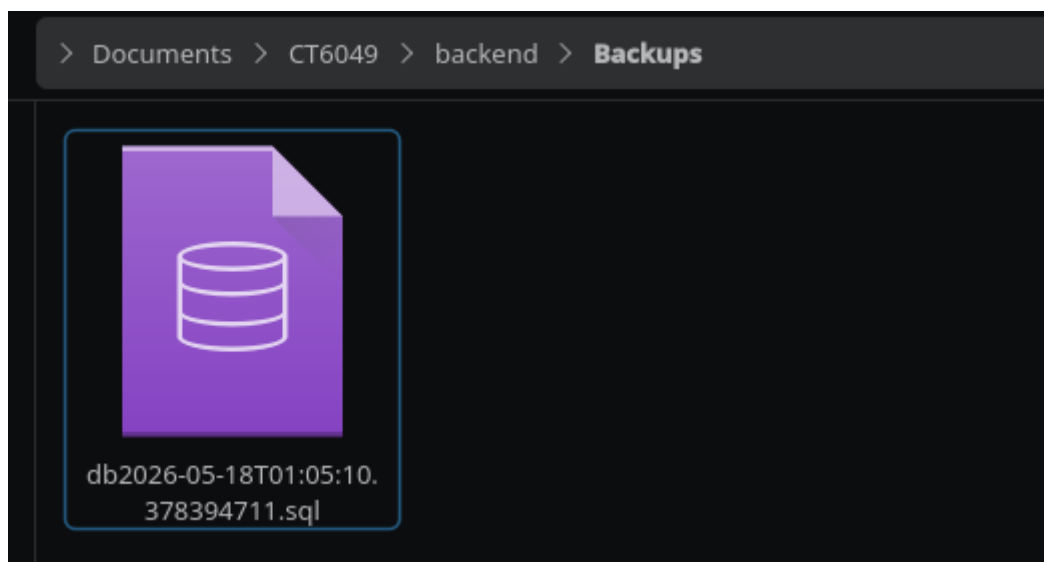


Figure 11: Backup Folder

To really ensure data preservation is adhered, Figure 12 shows the frontend handler function for restoring from a backup. Two addition process are involved:

1. Firstly, a single confirmation message box is displayed, requiring atleast 2 clicks. This confirmation box warns about the overwriting nature of restoring.
2. Secondly, and more importantly, a fresh backup is taken before the restoration occurs. There is no legitimate way for a admin to restore from a backup without first backing up the current state of the deployed database.

```
1  async function handleRestore(filename) {      ■ Parameter 'filename' implicitl
2      if (
3          !confirm(
4              'Restore from {filename}? This will overwrite the current data'.
5          )
6      )
7          return;
8      restoringFile = filename;
9      try {
10         handleCreate();
11
12         await restoreBackup(filename);
13         addAlert("Restored from previous backup");
14     } catch (err) {
15         addAlert(err.message);
16     } finally {
17         restoringFile = null;
18     }
19 }
```

Figure 12: Frontend restore backup handler

Bibliography

Amani Undru (2025) *Star Schema vs Snowflake Schema: 6 Key Differences*. Available at: <https://www.thoughtspot.com/data-trends/data-modeling/star-schema-vs-snowflake-schema> (Accessed: April 23, 2026).

Anna Litvinska (2025) *Operational Database vs. Data Warehouse: 7 Key Differences*. Available at: <https://www.cdata.com/blog/operational-database-vs-data-warehouse> (Accessed: April 28, 2026).

Augustine, J. (2024) *PostgreSQL Indexes Can Hurt Performance: Exploring the Negative Effects and the Costs Involved*. Available at: <https://www.percona.com/blog/postgresql-indexes-can-hurt-you-negative-effects-and-the-costs-involved/> (Accessed: May 5, 2026).

Charlie Custer (2023) *What is data partitioning, and how to do it right*. Available at: <https://www.cockroachlabs.com/blog/what-is-data-partitioning-and-how-to-do-it-right/> (Accessed: May 15, 2026).

itsimiro (2025) *Indexes in PostgreSQL: How They Work*. Available at: <https://itsimiro.medium.com/indexes-in-postgresql-how-they-work-2b98bf008020> (Accessed: May 11, 2026).

Laiba Siddiqui (2026) *Star Schema vs Snowflake Schema: Differences & Use Cases*. Available at: <https://www.datacamp.com/blog/star-schema-vs-snowflake-schema> (Accessed: April 23, 2026).

Lindsay MacDonald (2025) *Data Warehouse Schemas: Meet The Big 3 Everyone's Using*. Available at: <https://www.montecarlodata.com/blog-data-warehouse-schemas/> (Accessed: April 12, 2026).

Oracle (2021) *What Is ETL? Your Guide to Data Integration Essentials*. Available at: <https://www.oracle.com/uk/integration/what-is-etl/> (Accessed: May 3, 2026).

Oracle (2023) *What Is a Data Warehouse?*. Available at: <https://www.oracle.com/uk/database/what-is-a-data-warehouse/> (Accessed: April 18, 2026).

OWASP (2025) *OWASP Top 10:2025*. Available at: <https://owasp.org/Top10/2025/> (Accessed: May 18, 2026).

PostgreSQL (2026) *pg_dump*. Available at: <https://www.postgresql.org/docs/18/app-pgdump.html> (Accessed: May 18, 2026).

Smallcombe, M. (2026) *ETL vs ELT: 5 Critical Differences*. Available at: <https://www.integrate.io/blog/etl-vs-elt/> (Accessed: May 13, 2026).

Appendices

Appendix A: Additional Information

A.1 Application login details

Below some login details for different staff accounts are provided. As student accounts are unchanged from CT6049 A1, those details are not included; however using adminer, getting a student username is trivial, the password for all student accounts is change.

Username	Password	Role
admin	password	ADMIN
staff1	change	DECISION_MAKER
staff2		
staff3		
staff4		
staff5		

Table 1: Application account details

A.2 Database details

Field	Value
Host	localhost
Port	5432
Name	libdb
User	libuser
Password	libpass

Table 2: Postgresql Database details

A.3 Deployment Steps

Deployment steps are covered a bit more in detail in the README provided in the source code.

```
# DBA2 Library System
# Assignment 2 Program
```

Requirements

- * Docker / Docker Compose
- * Java 17
- * Postgresql 18
- * Npm / Node.JS

How to run

1. Start the development servers

```
```cmd
./run.sh or .\run.bat
```
```

The supplied run scripts *should* work. However if they do fail:

- Install frontend dependencies (cd frontend && npm install)
- Spin up docker containers (docker compose up -d)
- Open a terminal each in `frontend/` and `backend/` and run the commands:
frontend: npm run dev
backend: mvn spring-boot:run

2. Populate operational data

The python script provided in scripts/data.py is the seeder for the operational database tables. Download the dependencies and run like any normal python script.

3. Dashboard

Navigate to localhost:5173

Log into the desired account. Account details are provided in the appendices section of the main report.

Depedning on what account you log into, different features will be available (student accounts are functionally identical to users in DBA1)

4. ETL

To casue an ETL event, an admin must manually initiate one. Log into the admin account (if not already) and click the `LOAD ETL` button.

Additional info

```
ss -lputn 'sport = :20550
```

Listing 5: README.md

Appendix B: Additional Images

B.1 Admin Panel

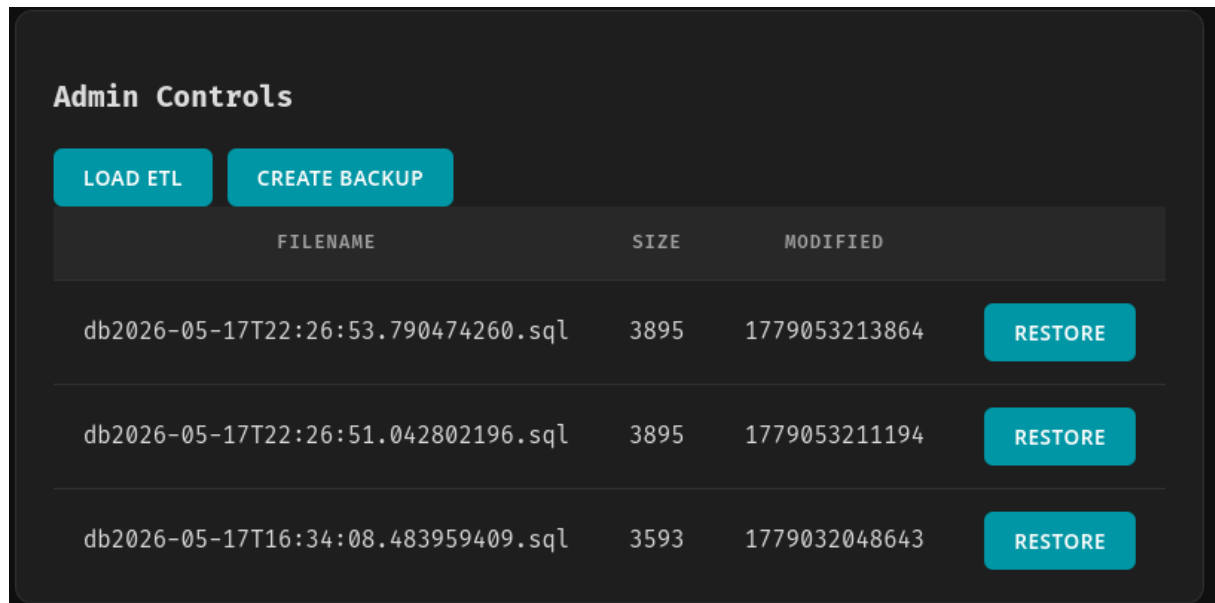


Figure 13: Admin panel showing ETL & Backup Buttons

Appendix C: Backend

C.1 Main App

```
package github.peaterpita;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class App {
    public static void main(String[] args) {
        SpringApplication.run(App.class, args);
    }
}
```

Listing 6: App.java

C.2 Redis Setup Configuration

```
package github.peaterpita.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.data.redis.connection.RedisConnectionFactory;
import org.springframework.data.redis.core.RedisTemplate;
import org.springframework.data.redis.serializer.RedisSerializer;
import org.springframework.data.redis.serializer.StringRedisSerializer;

@Configuration
public class redisConfig {

    // #####
    // # Defining new redis configuration so that image data #
    // # (byte arrays) can be cached for 24 hours #
    // # This is used in the caching of cover images #
    // #####
    @Bean
    public RedisTemplate<String, byte[]> redisTemplate(
        RedisConnectionFactory connectFac) {
        RedisTemplate<String, byte[]> template = new RedisTemplate<>();
        template.setConnectionFactory(connectFac);

        template.setKeySerializer(new StringRedisSerializer());
        template.setValueSerializer(RedisSerializer.byteArray());

        return template;
    }
}
```

Listing 7: redisConfig.java

C.3 Controller - Authentication

```
package github.peaterpita.controller;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.Map;
import github.peaterpita.service.AuthService;

@RestController
@RequestMapping("/api/auth")
public class AuthController {
    private final AuthService authService;

    public AuthController(AuthService authService) {
        this.authService = authService;
    }

    // #####
    // # Creating API endpoint `/api/auth/login`
    // # Login function takes two inputs, both present within
    // # the payload
    // # username and password
    // #
    // # Use authService attemptLogin function. If login
    // # successfull return success message with token
    // # else return 401 error
    // #####
    @PostMapping("/login")
    public ResponseEntity<?> login(
        @RequestBody Map<String, String> payload) {
        String username = payload.getOrDefault("username", "");
        String password = payload.getOrDefault("password", "");
        String token = authService.attemptLogin(username, password);

        ResponseEntity<?> res;
        if (token != null) {
            res = ResponseEntity.ok(Map.of(
                "success", true,
                "token", token));
        } else {
            res = ResponseEntity.status(401).body(Map.of(
                "success", false,
                "message", "Invalid username or password"));
        }
        return res;
    }
}
```

Listing 8: AuthController.java

C.4 Controller - Backup

```

package github.peaterpita.controller;

import java.util.Map;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import github.peaterpita.service.BackupService;

@RestController
@RequestMapping("/api/backup")
public class BackupController {

    private final BackupService backupService;

    public BackupController(BackupService backupService) {
        this.backupService = backupService;
    }

    // #####
    // # /api/backup GET
    // # Return a list of all the backups available
    // #####
    @GetMapping()
    public ResponseEntity<?> listBackups() {
        return ResponseEntity.ok(backupService.listBackups());
    }

    // #####
    // # /api/backup POST
    // # Create a new backup file using the service and return filename
    // #####
    @PostMapping()
    public ResponseEntity<?> createBackup() {
        try {
            String filename = backupService.createBackup();
            return ResponseEntity.ok(Map.of("success", true, "filename", filename));

        } catch (Exception err) {
            return ResponseEntity.internalServerError().body(Map.of("error",
err.getMessage()));
        }

    }

    // #####
    // # /api/backup/activate/{id}
    // # Restore database to a previous backup
    // #####
    @PostMapping("/activate/{filename}")
    public ResponseEntity<?> activateBackup(@PathVariable String filename) {
        try {
            Listing 9: BackupController.java
            backupService.activateBackup(filename);
            return ResponseEntity.ok(Map.of("success", true));
        } catch (Exception err) {
            return ResponseEntity.internalServerError().body(Map.of("error",
err.getMessage()));
        }
    }

```

C.5 Controller - Book

```

package github.peaterpita.controller;

import java.net.URI;
import java.util.List;
import java.util.stream.Collectors;

import org.springframework.http.MediaType;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

import github.peaterpita.dto.BookDto;
import github.peaterpita.model.Book;
import github.peaterpita.service.BookService;
import github.peaterpita.service.CoverService;

@RestController
@RequestMapping("/api/books")
public class BookController {
    private final BookService bookService;
    private final CoverService coverService;

    public BookController(
        BookService bookService,
        CoverService coverService) {
        this.bookService = bookService;
        this.coverService = coverService;
    }

    // #####
    // # /api/books
    // # Get back all books that fulfill search query
    // # and available check
    // #####
    @GetMapping
    public ResponseEntity<List<BookDto>> searchBooks(
        @RequestParam(value = "query", required = false) String query,
        @RequestParam(value = "available", required = false) Boolean available) {

        List<Book> results = bookService.searchBooks(query, available);
        List<BookDto> dtos = results.stream().map(BookDto::toDto)
            .collect(Collectors.toList());
        return ResponseEntity.ok(dtos);
    }

    // #####
    // # /api/books/{bookId}
    // # use bookService to find a book with the passed in ID
    // # if book found return it
    // # else send error
    // #####
    @GetMapping("/{id}")
    public ResponseEntity<BookDto> getBook(@PathVariable String id) {
        Listing 10: BookController.java
        try {
            Book book = bookService.findById(id);
            return ResponseEntity.ok(BookDto.toDto(book));
        } catch (RuntimeException e) {
            return ResponseEntity.notFound().build();
        }
    }

```

C.6 Controller = ETL

```
package github.peaterpita.controller;

import java.util.Map;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import github.peaterpita.service.EtlService;

@RestController
@RequestMapping("/api/etl")
public class EtlController {

    private final EtlService etlService;

    public EtlController(EtlService etlService) {
        this.etlService = etlService;
    }

    // #####
    // # /api/etl
    // # Endpoint to run the entire ETL chain of
    // # Loading data into warehouse dimensional tables
    // #####
    @PostMapping
    public ResponseEntity<?> run() {
        try {
            Map<String, Integer> result = etlService.run();
            return ResponseEntity.ok(result);
        } catch (Exception err) {
            return ResponseEntity.internalServerError().body(Map.of("error",
err.getMessage()));
        }
    }
}
```

Listing 11: EtlController.java

C.7 Controller - Loan

```

package github.peaterpita.controller;

import java.time.LocalDate;
import java.util.List;
import java.util.Map;

import org.apache.coyote.Response;
import org.springframework.http.ResponseEntity;
import org.springframework.security.core.Authentication;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

import github.peaterpita.dto.LoanDto;
import github.peaterpita.model.Loan;
import github.peaterpita.model.User;
import github.peaterpita.repository.UserRepository;
import github.peaterpita.service.LoanService;

@RestController
@RequestMapping("/api/loans")
public class LoanController {
    private final LoanService loanService;
    private final UserRepository userRepo;

    public LoanController(
        LoanService loanService,
        UserRepository userRepository) {
        this.loanService = loanService;
        this.userRepo = userRepository;
    }

    // #####
    // # Helper function to get currently signed in user through
    // # their JWT token.
    // # JWT token can be reversed to get the users username.
    // # This username can then be used
    // # to get overall user object
    // #####
    private User getCurrentUser(Authentication auth) {
        String username = (String) auth.getPrincipal();
        return userRepo.findByUsername(username).orElseThrow();
    }

    // #####
    // # Return back all loans associated with `me` (currently
    // # logged in user)
    // #####
    @GetMapping("/me")
    public ResponseEntity<List<LoanDto>> myLoans(Authentication auth) {
        User user = getCurrentUser(auth);
        List<LoanDto> loans = loanService.findLoansByUser(user.getId());
        return ResponseEntity.ok(loans);
    }

    @PostMapping("/borrow")
    public ResponseEntity<?> borrow(
        @RequestBody Map<String, Object> body,
        Authentication auth) {

```

Listing 12: LoanController.java

C.8 Controller - Warehousing

```

package github.peaterpita.controller;

import java.time.LocalDate;
import java.util.List;
import java.util.Map;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

import github.peaterpita.service.WarehouseService;

@RestController
@RequestMapping("/api/warehouse")
public class WarehouseController {
    private final WarehouseService warehouseService;

    public WarehouseController(WarehouseService warehouseService) {
        this.warehouseService = warehouseService;
    }

    @GetMapping("/loans/trend")
    public ResponseEntity<List<Map<String, Object>>> loanTrend(
        @RequestParam(required = false) LocalDate start,
        @RequestParam(required = false) LocalDate end) {
        return ResponseEntity.ok(warehouseService.getLoanTrend(start, end));
    }

    @GetMapping("/fines/trend")
    public ResponseEntity<List<Map<String, Object>>> fineRevenue(
        @RequestParam(required = false) LocalDate start,
        @RequestParam(required = false) LocalDate end) {
        return ResponseEntity.ok(warehouseService.getFineRevenue(start, end));
    }

    @GetMapping("/loans/overdue")
    public ResponseEntity<List<Map<String, Object>>> overdue() {
        return ResponseEntity.ok(warehouseService.getOverdue());
    }

    @GetMapping("/books/popular")
    public ResponseEntity<List<Map<String, Object>>> popular(
        @RequestParam(required = false) LocalDate start,
        @RequestParam(required = false) LocalDate end) {
        return ResponseEntity.ok(warehouseService.getPopular(start, end));
    }

    @GetMapping("/loans/duration")
    public ResponseEntity<List<Map<String, Object>>> duration(
        @RequestParam(required = false) LocalDate start,
        @RequestParam(required = false) LocalDate end) {
        return ResponseEntity.ok(warehouseService.getLoanDuration(start, end));
    }

    @GetMapping("/loans/course")
    public ResponseEntity<List<Map<String, Object>>> courseEngagment(
        @RequestParam(required = false) LocalDate start,
        @RequestParam(required = false) LocalDate end) {
        return ResponseEntity.ok(warehouseService.getCourseEngagment(start, end));
    }
}

```

Listing 16: WarehouseController.java

C.9 DTO - Book

```
package github.peaterpita.dto;

import github.peaterpita.model.Book;

public class BookDto {
    public String id;
    public String isbn;
    public String title;
    public String author;
    public String coverURL;
    public Integer copiesLeft;

    public static BookDto toDto(Book book) {
        BookDto dto = new BookDto();

        dto.id = book.getId();
        dto.isbn = book.getIsbn();
        dto.title = book.getTitle();
        dto.author = book.getAuthor();
        dto.copiesLeft = book.getCopiesLeft();

        // #####
        // # If no ISBN present (never should occur)
        // # set coverURL to a placeholder image stored on the
        // # frontend
        // #####
        if (book.getIsbn() != null) {
            dto.coverURL = "/api/books/cover/" + book.getIsbn();
        } else {
            dto.coverURL = "/assets/placeholder.png";
        }

        return dto;
    }
}
```

Listing 14: BookDto.java

C.10 DTO - Loan

```
package github.peaterpita.dto;

import java.time.LocalDate;

public class LoanDto {
    public String id;
    public BookDto book;
    public LocalDate loanDate;
    public LocalDate dueDate;
    public LocalDate returnDate;
    public String status;
    public Double fine;
}
```

Listing 15: LoanDto.java

C.11 Model - Book

```
package github.peaterpita.model;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name = "books")
public class Book {

    @Id
    private String id;

    @Column(unique = true)
    private String isbn;

    @Column(nullable = false)
    private String title;

    @Column(nullable = false)
    private String author;

    @Column(name = "copies_left")
    private Integer copiesLeft = 1;

    public String getId() { return id; }
    public void setId(String id) { this.id = id; }

    public String getIsbn() { return isbn; }
    public void setIsbn(String isbn) { this.isbn = isbn; }

    public String getTitle() { return title; }
    public void setTitle(String title) { this.title = title; }

    public String getAuthor() { return author; }
    public void setAuthor(String author) { this.author = author; }

    public Integer getCopiesLeft() { return copiesLeft; }
    public void setCopiesLeft(Integer copiesLeft) { this.copiesLeft = copiesLeft; }
}
```

Listing 16: Book.java

C.12 Model - Course

```
package github.peaterpita.model;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name = "courses")
public class Course {

    @Id
    @Column(name = "course_id")
    private String courseId;

    @Column(name = "course_name", nullable = false)
    private String courseName;

    @Column(name = "course_code", unique = true, nullable = false)
    private String courseCode;

    public String getCourseId() { return courseId; }
    public void setCourseId(String courseId) { this.courseId = courseId; }

    public String getCourseName() { return courseName; }
    public void setCourseName(String courseName) { this.courseName = courseName; }

    public String getCourseCode() { return courseCode; }
    public void setCourseCode(String courseCode) { this.courseCode = courseCode; }
}
```

Listing 17: Course.java

C.13 Model - Loan

```

package github.peaterpita.model;

import java.time.LocalDate;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name = "loans")
public class Loan {

    @Id
    private String id;

    @Column(name = "user_id", nullable = false)
    private String userId;

    @Column(name = "book_id", nullable = false)
    private String bookId;

    @Column(name = "loan_date")
    private LocalDate loanDate;

    @Column(name = "due_date")
    private LocalDate dueDate;

    @Column(name = "return_date")
    private LocalDate returnDate;

    private String status;

    @Column(name = "fine_amount")
    private Double fineAmount;

    public String getId() { return id; }
    public void setId(String id) { this.id = id; }

    public String getUserId() { return userId; }
    public void setUserId(String userId) { this.userId = userId; }

    public String getBookId() { return bookId; }
    public void setBookId(String bookId) { this.bookId = bookId; }

    public LocalDate getLoanDate() { return loanDate; }
    public void setLoanDate(LocalDate loanDate) { this.loanDate = loanDate; }

    public LocalDate getDueDate() { return dueDate; }
    public void setDueDate(LocalDate dueDate) { this.dueDate = dueDate; }

    public LocalDate getReturnDate() { return returnDate; }
    public void setReturnDate(LocalDate returnDate) { this.returnDate = returnDate; }

    public String getStatus() { return status; }
    public void setStatus(String status) { this.status = status; }

    public Double getFineAmount() { return fineAmount; }
    public void setFineAmount(Double fineAmount) { this.fineAmount = fineAmount; }
}

```

Listing 18: Loan.java

C.14 Model - Module

```
package github.peaterpita.model;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name = "modules")
public class Module {

    @Id
    @Column(name = "module_id")
    private String moduleId;

    @Column(name = "module_name", nullable = false)
    private String moduleName;

    @Column(name = "module_code", unique = true, nullable = false)
    private String moduleCode;

    public String getModuleId() { return moduleId; }
    public void setModuleId(String moduleId) { this.moduleId = moduleId; }

    public String getModuleName() { return moduleName; }
    public void setModuleName(String moduleName) { this.moduleName = moduleName; }

    public String getModuleCode() { return moduleCode; }
    public void setModuleCode(String moduleCode) { this.moduleCode = moduleCode; }
}
```

Listing 19: Module.java

C.15 Model - Staff

```
package github.peaterpita.model;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name = "staff")
public class Staff {

    @Id
    @Column(name = "staff_id")
    private String staffId;

    @Column(name = "user_id", nullable = false, unique = true)
    private String userId;

    @Column(nullable = false)
    private String role;

    public String getStaffId() { return staffId; }
    public void setStaffId(String staffId) { this.staffId = staffId; }

    public String getUserId() { return userId; }
    public void setUserId(String userId) { this.userId = userId; }

    public String getRole() { return role; }
    public void setRole(String role) { this.role = role; }
}
```

Listing 20: Staff.java

C.16 Model - Student

```
package github.peaterpita.model;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name = "students")
public class Student {

    @Id
    @Column(name = "student_id")
    private String studentId;

    @Column(name = "user_id", nullable = false, unique = true)
    private String userId;

    @Column(name = "student_number", unique = true, nullable = false)
    private String studentNumber;

    @Column(name = "course_id", nullable = false)
    private String courseId;

    public String getStudentId() { return studentId; }
    public void setStudentId(String studentId) { this.studentId = studentId; }

    public String getUserId() { return userId; }
    public void setUserId(String userId) { this.userId = userId; }

    public String getStudentNumber() { return studentNumber; }
    public void setStudentNumber(String studentNumber) { this.studentNumber =
studentNumber; }

    public String getCourseId() { return courseId; }
    public void setCourseId(String courseId) { this.courseId = courseId; }
}
```

Listing 21: Student.java

C.17 Model - User

```
package github.peaterpita.model;

import java.time.LocalDateTime;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name = "users")
public class User {

    @Id
    private String id;

    @Column(unique = true, nullable = false)
    private String username;

    @Column(name = "password_hash", nullable = false)
    private String passwordHash;

    @Column(name = "created_at")
    private LocalDateTime createdAt;

    @Column(name = "last_login")
    private LocalDateTime lastLogin;

    public String getId() { return id; }
    public void setId(String id) { this.id = id; }

    public String getUsername() { return username; }
    public void setUsername(String username) { this.username = username; }

    public String getPasswordHash() { return passwordHash; }
    public void setPasswordHash(String passwordHash) { this.passwordHash =
passwordHash; }

    public LocalDateTime getCreatedAt() { return createdAt; }
    public void setCreatedAt(LocalDateTime createdAt) { this.createdAt = createdAt; }

    public LocalDateTime getLastLogin() { return lastLogin; }
    public void setLastLogin(LocalDateTime lastLogin) { this.lastLogin = lastLogin; }
}
```

Listing 22: User.java

C.18 Repository - Book

```
package github.peaterpita.repository;

import java.util.List;

import org.springframework.data.jpa.repository.JpaRepository;

import github.peaterpita.model.Book;

public interface BookRepository extends JpaRepository<Book, String> {

    List<Book> findByTitleContainingIgnoreCase(String title);

    List<Book> findByAuthorContainingIgnoreCase(String author);

    List<Book> findByCopiesLeftGreaterThan(int copies);
}
```

Listing 23: BookRepository.java

C.19 Repository - Course

```
package github.peaterpita.repository;

import org.springframework.data.jpa.repository.JpaRepository;

import github.peaterpita.model.Course;

public interface CourseRepository extends JpaRepository<Course, String> {
}
```

Listing 24: CourseRepository.java

C.20 Repository - Loan

```
package github.peaterpita.repository;

import java.time.LocalDate;
import java.util.List;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;

import github.peaterpita.model.Loan;

public interface LoanRepository extends JpaRepository<Loan, String> {

    List<Loan> findByUserId(String userId);

    @Query("SELECT l FROM Loan l WHERE l.userId = :userId " +
        "AND l.loanDate BETWEEN :begin AND :end")
    List<Loan> findHistoryBetween(
        @Param("userId") String userId,
        @Param("begin") LocalDate begin,
        @Param("end") LocalDate end);

    boolean existsByUserIdAndBookIdAndReturnDateIsNull(String userId, String bookId);
}
```

Listing 25: LoanRepository.java

C.21 Repository - Module

```
package github.peaterpita.repository;

import org.springframework.data.jpa.repository.JpaRepository;

import github.peaterpita.model.Module;

public interface ModuleRepository extends JpaRepository<Module, String> {
}
```

Listing 26: ModuleRepository.java

C.22 Repository - Staff

```
package github.peaterpita.repository;

import java.util.Optional;

import org.springframework.data.jpa.repository.JpaRepository;

import github.peaterpita.model.Staff;

public interface StaffRepository extends JpaRepository<Staff, String> {

    Optional<Staff> findByUserId(String userId);
}
```

Listing 27: StaffRepository.java

C.23 Repository - Student

```
package github.peaterpita.repository;

import java.util.Optional;

import org.springframework.data.jpa.repository.JpaRepository;

import github.peaterpita.model.Student;

public interface StudentRepository extends JpaRepository<Student, String> {

    Optional<Student> findByUserId(String userId);
}
```

Listing 28: StudentRepository.java

C.24 Repository - User

```
package github.peaterpita.repository;

import java.util.Optional;

import org.springframework.data.jpa.repository.JpaRepository;

import github.peaterpita.model.User;

public interface UserRepository extends JpaRepository<User, String> {

    Optional<User> findByUsername(String username);
}
```

Listing 29: UserRepository.java

C.25 Security - JWT Filter

```
package github.peaterpita.security;

import java.io.IOException;

import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.util.StringUtils;
import org.springframework.web.filter.OncePerRequestFilter;

import java.util.List;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

public class JwtAuthenticationFilter extends OncePerRequestFilter {
    private final JwtUtil jwtUtil;

    public JwtAuthenticationFilter(JwtUtil jwtUtil) {
        this.jwtUtil = jwtUtil;
    }

    @Override
    protected void doFilterInternal(
        HttpServletRequest request,
        HttpServletResponse response,
        FilterChain filterChain)
        throws ServletException, IOException {

        String header = request.getHeader("Authorization");
        String token = null;

        if (StringUtils.hasText(header) && header.startsWith("Bearer ")) {
            token = header.substring(7);
        }

        if (token != null && jwtUtil.validateToken(token)) {
            String username = jwtUtil.getUsernameFromToken(token);
            String role = jwtUtil.getRoleFromToken(token);

            var auth = new UsernamePasswordAuthenticationToken(username, null,
                List.of(new SimpleGrantedAuthority("ROLE_" + role)));
            SecurityContextHolder.getContext().setAuthentication(auth);
        }

        filterChain.doFilter(request, response);
    }
}
```

Listing 30: JwtAuthenticationFilter.java

C.26 Security - JWT Utilities

```

package github.peaterpita.security;

import java.security.Key;
import java.util.Date;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;

import io.jsonwebtoken.JwtException;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.security.Keys;

@Component
public class JwtUtil {

    private final Key key;
    private final long expirationMs;

    public JwtUtil(
        @Value("${jwt.secret}") String secret,
        @Value("${jwt.expiration-ms}") long expirationMsEnv) {
        this.key = Keys.hmacShaKeyFor(secret.getBytes());
        this.expirationMs = expirationMsEnv;
    }

    public String generateToken(String username, String role) {
        Date now = new Date();
        Date expir = new Date(now.getTime() + expirationMs);

        return Jwts.builder()
            .setSubject(username)
            .claim("role", role)
            .setIssuedAt(now)
            .setExpiration(expir)
            .signWith(key, io.jsonwebtoken.SignatureAlgorithm.HS256)
            .compact();
    }

    public String getRoleFromToken(String token) {
        return Jwts.parserBuilder()
            .setSigningKey(key)
            .build()
            .parseClaimsJws(token)
            .getBody()
            .get("role")
            .toString();
    }

    public String getUsernameFromToken(String token) throws JwtException {
        return Jwts.parserBuilder()
            .setSigningKey(key)
            .build()
            .parseClaimsJws(token)
            .getBody()
            .getSubject();
    }
}

```

Listing 31: JwtUtil.java

```

public boolean validateToken(String token) {
    try {
        Jwts.parserBuilder().setSigningKey(key).build().parseClaimsJws(token);
        return true;
    } catch (JwtException e) {
        return false;
    }
}

```

C.27 Security - Configuration

```

package github.peaterpita.security;

import java.util.List;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import
org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
import org.springframework.web.cors.CorsConfiguration;

@Configuration
public class SecurityConfig {

    private final JwtUtil jwtUtil;
    private final List<String> allowedOrigins;

    public SecurityConfig(JwtUtil jwtUtil,
        @Value("${cors.allowed-origins}") List<String> allowedOrigins) {
        this.jwtUtil = jwtUtil;
        this.allowedOrigins = allowedOrigins;
    }

    @Bean
    public BCryptPasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        var jwtFilter = new JwtAuthenticationFilter(jwtUtil);

        http.csrf(csrf -> csrf.disable())
            .cors(cors -> cors.configurationSource(request -> {
                var cfg = new CorsConfiguration();
                cfg.setAllowedOrigins(allowedOrigins);
                cfg.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE",
"OPTIONS"));
                cfg.setAllowedHeaders(List.of("*"));
                cfg.setAllowCredentials(true);
                return cfg;
            })))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/**", "/").permitAll()
                .requestMatchers(HttpMethod.GET, "/api/books/cover/
**").permitAll()
                .requestMatchers("/api/warehouse/
**").hasAnyRole("DECISION_MAKER", "ADMIN")
                .requestMatchers("/api/etl/**").hasRole("ADMIN")
                .requestMatchers("/api/backup/**").hasRole("ADMIN")
                .anyRequest().authenticated())
            .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class);
        // .httpBasic(Customizer.withDefaults());

        return http.build();
    }
}

```

Listing 52: SecurityConfig.java

C.28 Service - Authentication

```
package github.peaterpita.service;

import java.util.Optional;

import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.stereotype.Service;

import github.peaterpita.model.Staff;
import github.peaterpita.model.User;
import github.peaterpita.repository.StaffRepository;
import github.peaterpita.repository.UserRepository;
import github.peaterpita.security.JwtUtil;

@Service
public class AuthService {
    private final UserRepository userRepo;
    private final StaffRepository staffRepo;
    private final BCryptPasswordEncoder passwordEncoder;
    private final JwtUtil jwtUtil;

    public AuthService(UserRepository userRepo, StaffRepository staffRepo,
BCryptPasswordEncoder passwordEncoder,
        JwtUtil jwtUtil) {
        this.userRepo = userRepo;
        this.staffRepo = staffRepo;
        this.passwordEncoder = passwordEncoder;
        this.jwtUtil = jwtUtil;
    }

    public String attemptLogin(String username, String password) {
        Optional<User> userOpt = userRepo.findByUsername(username);

        if (userOpt.isEmpty())
            return null;

        User user = userOpt.get();

        if (!passwordEncoder.matches(password, user.getPasswordHash()))
            return null;

        String role =
staffRepo.findByUserId(user.getId()).map(Staff::getRole).orElse("STUDENT");
        return jwtUtil.generateToken(user.getUsername(), role);
    }
}
```

Listing 33: AuthService.java

C.29 Service - Backup

```

package github.peaterpita.service;

import java.io.File;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.time.LocalDateTime;
import java.util.Arrays;
import java.util.List;
import java.util.Map;

import org.springframework.stereotype.Service;

@Service
public class BackupService {

    private String backupPath = "./Backups/";

    private void ensureBackupDir() {
        File dir = new File(backupPath);
        if (!dir.exists())
            dir.mkdirs();
    }

    public List<Map<String, Object>> listBackups() {
        ensureBackupDir();

        File dir = new File(backupPath);
        File[] files = dir.listFiles((unused, name) -> name.endsWith(".sql"));

        if (files == null)
            return List.of();

        return Arrays.stream(files)
            .sorted((a, b) -> Long.compare(b.lastModified(), a.lastModified()))
            .map(file -> Map.<String, Object>.of(
                "filename", file.getName(),
                "size", file.length() / 1204,
                "modified", file.lastModified()))
            .toList();
    }

    public String createBackup() {
        ensureBackupDir();
        String filename = "db" + LocalDateTime.now().toString() + ".sql";
        Path target = Paths.get(backupPath, filename);

        ProcessBuilder process = new ProcessBuilder(
            "pg_dump",
            "-h", "localhost",
            "-p", "5432",
            "-U", "libuser",
            "-d", "libdb",
            "-f", target.toString());

        process.environment().put("PGPASSWORD", "libuser");
        process.redirectErrorStream(true);

        try {
            Process proc = process.start();
            int exitCode = proc.waitFor();
        } catch (IOException e) {
            // ...
        }
    }
}

```

Listing 34. BackupService.java

C.30 Service - Book

```
package github.peaterpita.service;

import java.util.List;
import java.util.stream.Collectors;

import org.springframework.stereotype.Service;

import github.peaterpita.model.Book;
import github.peaterpita.repository.BookRepository;

@Service
public class BookService {
    private final BookRepository bookRepo;

    public BookService(BookRepository bookRepo) {
        this.bookRepo = bookRepo;
    }

    public List<Book> findAll() {
        return bookRepo.findAll();
    }

    public Book findById(String id) {
        return bookRepo.findById(id)
            .orElseThrow(() -> new RuntimeException("Book not found"));
    }

    public List<Book> getAvailable() {
        return bookRepo.findByCopiesLeftGreaterThan(0);
    }

    public List<Book> searchBooks(String query, Boolean available) {
        List<Book> results = bookRepo.findAll();

        if (query != null && !query.isBlank()) {
            String lower = query.toLowerCase();
            results = results.stream()
                .filter(b -> b.getTitle().toLowerCase().contains(lower)
                    || b.getAuthor().toLowerCase().contains(lower))
                .collect(Collectors.toList());
        }

        if (Boolean.TRUE.equals(available)) {
            results = results.stream()
                .filter(b -> b.getCopiesLeft() != null && b.getCopiesLeft() > 0)
                .collect(Collectors.toList());
        }

        return results;
    }
}
```

Listing 35: BookService.java

C.31 Service - Cover

```

package github.peaterpita.service;

import java.time.Duration;

import org.springframework.data.redis.core.RedisTemplate;
import org.springframework.http.ResponseEntity;
import org.springframework.stereotype.Service;
import org.springframework.web.client.HttpClientErrorException;
import org.springframework.web.client.RestTemplate;

@Service
public class CoverService {
    private final RedisTemplate<String, byte[]> imgTemplate;
    private final RestTemplate restTemplate;

    public CoverService(RedisTemplate<String, byte[]> imgTemplate) {
        this.imgTemplate = imgTemplate;
        this.restTemplate = new RestTemplate();
    }

    public byte[] getCover(String isbn) {

        if (isbn == null || isbn.isBlank()) {
            return null;
        }

        String cacheKey = "cover:" + isbn;

        // #####
        // # Check if cover is cached in Redis first. If it is use
        // # that
        // #####
        if (Boolean.TRUE.equals(imgTemplate.hasKey(cacheKey))) {
            return imgTemplate.opsForValue().get(cacheKey);
        }

        try {

            // #####
            // # If the cover is not cached, reach out to the
            // # openlibrary covers api to get a cover for that isbn
            // # If a reply is recieved and the headers are good
            // # status, extract body
            // # and cache contents into redis.
            // # return same contents out
            // #####
            String url = "https://covers.openlibrary.org/b/isbn/"
                + isbn
                + "-M.jpg?default=false";
            ResponseEntity<byte[]> res = restTemplate.getForEntity(url,
byte[].class);

            if (res.getStatusCode().is2xxSuccessful() && res.getBody() != null) {
                byte[] imgBytes = res.getBody();
                imgTemplate.opsForValue().set(cacheKey, imgBytes,
Duration.ofHours(96));
                return imgBytes;
            }
        } catch (HttpClientErrorException.NotFound err) {
            // #####
            // # Even if the request to OpenLibrary fails,
            // # still cache a null. This prevents us from that

```

Listing 36: CoverService.java

C.32 Service - ETL

```

package github.peaterpita.service;

import java.util.HashMap;
import java.util.Map;

import org.springframework.stereotype.Service;

import jakarta.persistence.EntityManager;
import jakarta.persistence.PersistenceContext;
import jakarta.transaction.Transactional;

@Service
public class EtlService {

    @PersistenceContext
    private EntityManager entityManager;

    @Transactional
    public Map<String, Integer> run() {
        loadDimDate();
        Map<String, Integer> result = new HashMap<>();
        result.put("courses", loadDimCourse());
        result.put("books", loadDimBook());
        result.put("students", loadDimStudent());
        result.put("loans", loadFactLoans());

        return result;
    }

    private int loadDimDate() {
        String query = """
            INSERT INTO warehouse.dim_date (full_date, day_of_week, day_name,
            week_of_year, month, month_name, year, is_weekend)
            SELECT DISTINCT
                loan_date,
                EXTRACT(DOW FROM loan_date)::INT,
                TO_CHAR(loan_date, 'FMDay'),
                EXTRACT(WEEK FROM loan_date)::INT,
                EXTRACT(MONTH FROM loan_date)::INT,
                TO_CHAR(loan_date, 'FMMONTH'),
                EXTRACT(YEAR FROM loan_date)::INT,
                EXTRACT(DOW FROM loan_date) IN (0, 6)
            FROM loans
            ON CONFLICT (full_date) DO NOTHING
            """;

        return entityManager.createNativeQuery(query).executeUpdate();
    }

    private int loadDimCourse() {
        String query = """
            INSERT INTO warehouse.dim_course (course_id, course_name,
            course_code)
            SELECT course_id, course_name, course_code
            FROM courses
            WHERE course_id NOT IN (
                SELECT course_id
                FROM warehouse.dim_course
            )
            """;

        return entityManager.createNativeQuery(query).executeUpdate();
    }
}

```

C.33 Service - Loan

```

package github.peaterpita.service;

import java.time.LocalDate;
import java.time.temporal.ChronoUnit;
import java.util.List;
import java.util.UUID;
import java.util.stream.Collectors;

import org.springframework.stereotype.Service;

import github.peaterpita.dto.BookDto;
import github.peaterpita.dto.LoanDto;
import github.peaterpita.model.Book;
import github.peaterpita.model.Loan;
import github.peaterpita.repository.BookRepository;
import github.peaterpita.repository.LoanRepository;

@Service
public class LoanService {
    private final LoanRepository loanRepo;
    private final BookRepository bookRepo;
    private static final double FINE_ADDITION = 1.50;

    public LoanService(LoanRepository loanRepo, BookRepository bookRepo) {
        this.loanRepo = loanRepo;
        this.bookRepo = bookRepo;
    }

    private LoanDto toDto(Loan loan) {
        LoanDto dto = new LoanDto();
        dto.id = loan.getId();
        dto.loanDate = loan.getLoanDate();
        dto.returnDate = loan.getReturnDate();
        dto.dueDate = loan.getDueDate();

        if (loan.getStatus().equals("BORROWED") &&
LocalDate.now().isAfter(loan.getDueDate())) {
            dto.status = "OVERDUE";
            long daysLate = ChronoUnit.DAYS.between(loan.getDueDate(),
LocalDate.now());
            dto.fine = daysLate * FINE_ADDITION;
        } else {
            dto.status = loan.getStatus();
            dto.fine = loan.getFineAmount();
        }

        if (loan.getBookId() != null) {
            bookRepo.findById(loan.getBookId()).ifPresent(book -> dto.book =
BookDto.toDto(book));
        }
        if (dto.book == null) {
            dto.book = new BookDto();
            dto.book.title = "Unknown";
            dto.book.author = "Unknown";
        }
        return dto;
    }
}

```

Listing 38: LoanService.java

```

public List<LoanDto> findLoansByUser(String userId) {
    return loanRepo.findById(userId)
        .stream().map(this::toDto).collect(Collectors.toList());
}

```

C.34 Service - Warehousing

```

package github.peaterpita.service;

import java.time.LocalDate;
import java.util.List;
import java.util.Map;
import java.util.stream.Collectors;

import org.springframework.stereotype.Service;

import jakarta.persistence.EntityManager;
import jakarta.persistence.PersistenceContext;

@Service
public class WarehouseService {

    @PersistenceContext
    private EntityManager entityManager;

    public List<Map<String, Object>> getLoanTrend(LocalDate start, LocalDate end) {
        String query = """
total_loans
            SELECT dimd.year, dimd.month, dimd.month_name, COUNT(*) AS
            FROM warehouse.fact_loans factl
            JOIN warehouse.dim_date dimd ON factl.date_key = dimd.date_key
            WHERE (CAST(:start AS DATE) IS NULL OR dimd.full_date >= :start)
            AND (CAST(:end AS DATE) IS NULL OR dimd.full_date <= :end)
            GROUP BY dimd.year, dimd.month, dimd.month_name
            ORDER BY dimd.year, dimd.month
            """;

        @SuppressWarnings("unchecked")
        List<Object[]> results = entityManager.createNativeQuery(query)
            .setParameter("start", start).setParameter("end", end)
            .getResultList();

        return results.stream().map(row -> Map.of(
            "year", row[0],
            "month", row[1],
            "monthName", row[2],
            "totalLoans", row[3])).collect(Collectors.toList());
    }

    public List<Map<String, Object>> getOverdue() {
        String query = """
            SELECT dimc.course_name, COUNT(*) AS overdue_count,
            COALESCE(SUM(factl.fine_amount), 0) AS total_fines
            FROM warehouse.fact_loans factl
            JOIN warehouse.dim_course dimc ON factl.course_key = dimc.course_key
            JOIN warehouse.dim_date dimd ON factl.date_key = dimd.date_key
            WHERE factl.overdue = true
            GROUP BY dimc.course_name
            ORDER BY overdue_count DESC
            """;

        @SuppressWarnings("unchecked")
        List<Object[]> results = entityManager.createNativeQuery(query)
            .getResultList();

        Listing 39: WarehouseService.java
        return results.stream().map(row -> Map.of(
            "courseName", row[0],
            "overdueCount", row[1],
            "totalFines", row[2])).collect(Collectors.toList());
    }
}

```

Appendix D: Migration

D.1 Operational Tables

```

CREATE TABLE IF NOT EXISTS users (
    id VARCHAR(36) PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    created_at TIMESTAMP,
    last_login TIMESTAMP
);

CREATE TABLE IF NOT EXISTS books (
    id VARCHAR(36) PRIMARY KEY,
    isbn VARCHAR(13) UNIQUE,
    title VARCHAR(255) NOT NULL,
    author VARCHAR(200) NOT NULL,
    copies_left INTEGER DEFAULT 1
);

CREATE TABLE IF NOT EXISTS loans (
    id VARCHAR(36) PRIMARY KEY,
    user_id VARCHAR(36) NOT NULL REFERENCES users (id),
    book_id VARCHAR(36) NOT NULL REFERENCES books (id),
    loan_date DATE,
    due_date DATE,
    return_date DATE,
    status VARCHAR(20),
    fine_amount DOUBLE PRECISION
);

CREATE TABLE IF NOT EXISTS courses (
    course_id VARCHAR(36) PRIMARY KEY,
    course_name VARCHAR(150) NOT NULL,
    course_code VARCHAR(20) UNIQUE NOT NULL
);

CREATE TABLE IF NOT EXISTS modules (
    module_id VARCHAR(36) PRIMARY KEY,
    module_name VARCHAR(150) NOT NULL,
    module_code VARCHAR(20) NOT NULL UNIQUE
);

CREATE TABLE IF NOT EXISTS course_modules (
    course_id VARCHAR NOT NULL REFERENCES courses (course_id),
    module_id VARCHAR NOT NULL REFERENCES modules (module_id),
    PRIMARY KEY (course_id, module_id)
);

CREATE TABLE IF NOT EXISTS students (
    student_id VARCHAR(36) PRIMARY KEY,
    user_id VARCHAR NOT NULL REFERENCES users (id),
    student_number VARCHAR(20) UNIQUE NOT NULL,
    course_id VARCHAR NOT NULL REFERENCES courses (course_id),
    study_year INT NOT NULL DEFAULT 1
);

CREATE TABLE IF NOT EXISTS staff (
    staff_id VARCHAR(36) PRIMARY KEY,
    user_id VARCHAR NOT NULL REFERENCES users (id),
    role VARCHAR(32) DEFAULT 'DECISION_MAKER' CHECK (
        role IN ('DECISION_MAKER', 'ADMIN')
    )
);

```

D.2 Warehousing Tables

```
CREATE SCHEMA IF NOT EXISTS warehouse;
```

```
CREATE TABLE IF NOT EXISTS warehouse.dim_date (  
date_key INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
full_date DATE NOT NULL UNIQUE,  
day_name VARCHAR (10) NOT NULL,  
month_name VARCHAR (10) NOT NULL,  
year INT NOT NULL,  
day_of_week INT NOT NULL,  
week_of_year INT NOT NULL,  
month INT NOT NULL,  
is_weekend BOOLEAN NOT NULL DEFAULT FALSE  
) ;
```

```
CREATE TABLE IF NOT EXISTS warehouse.dim_book (  
book_key INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
book_id VARCHAR (36) NOT NULL,  
isbn VARCHAR (36),  
title VARCHAR (255),  
author VARCHAR (255),  
effective_from DATE NOT NULL,  
effective_to DATE,  
is_current BOOLEAN NOT NULL DEFAULT TRUE  
) ;
```

```
CREATE TABLE IF NOT EXISTS warehouse.dim_course (  
course_key INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
course_id VARCHAR (36) NOT NULL,  
course_name VARCHAR (150),  
course_code VARCHAR (20)  
) ;
```

```
CREATE TABLE IF NOT EXISTS warehouse.dim_student (  
student_key INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
student_id VARCHAR (36) NOT NULL,  
student_number VARCHAR (20),  
course_key INT REFERENCES warehouse.dim_course (course_key),  
study_year INT NOT NULL,  
effective_from DATE NOT NULL,  
effective_to DATE,  
is_current BOOLEAN NOT NULL DEFAULT TRUE  
) ;
```

```
CREATE TABLE IF NOT EXISTS warehouse.fact_loans (  
loan_key INT GENERATED BY DEFAULT AS IDENTITY,  
loan_id VARCHAR (36) NOT NULL,  
date_key INT NOT NULL REFERENCES warehouse.dim_date (date_key),  
book_key INT NOT NULL REFERENCES warehouse.dim_book (book_key),  
student_key INT NOT NULL REFERENCES warehouse.dim_student (student_key),  
course_key INT REFERENCES warehouse.dim_course (course_key),  
loan_duration INT,  
overdue BOOLEAN NOT NULL DEFAULT FALSE,  
fine_amount DOUBLE PRECISION,  
status VARCHAR (20),  
loan_year INT NOT NULL,
```

Listing 41: V2__warehouse.sql

xlix

```
CONSTRAINT fact_loans_id UNIQUE (loan_id, loan_year)  
) PARTITION BY RANGE (loan_year) ;
```

D.3 Operational Data Seeder

```

import random
import bcrypt
import uuid
from datetime import date, timedelta
from faker import Faker
import psychpg2
from psychpg2.extras import execute_values

```

```

fake = Faker("en_GB")

```

```

COURSE_TEMPLATES = [
    ("Computer Science", "CS"),
    ("Data Science", "DS"),
    ("Cyber Security", "CY"),
    ("Accounting And Bussiness Management", "AB"),
    ("Financial Technology", "FT"),
    ("Animation", "AA"),
    ("Games Art", "GA"),
    ("Graphic Design", "GD"),
    ("Computer Games Programming", "GP"),
    ("Marketing", "MA"),
    ("Marketing (Digital Innovation)", "MI"),
    ("Physcial Education", "PE"),
    ("Sport Business Management", "SM"),
    ("Sports Journalism", "SJ"),
]

```

```

MODULE_TEMPLATES = [
    ("Distributed Databases and Data Warehousing", "CT6049"),
    ("Malware Analysis", "CT6044"),
    ("Advanced Networking", "CT6034"),
    ("Dissertation", "CT6039"),
    ("Secure Coding", "CT6042"),
    ("Cyber Law", "BM7214"),
    ("Business Development Project", "BM7111"),
    ("Big Data in Finance", "FT7102"),
    ("Artificial Intelligence", "FT7206"),
    ("Block Chain And Applications", "FT7103"),
    ("Introduction to Law", "AC4104"),
    ("Fundamentals Of Financial Accouting", "AC4101"),
    ("Introduction to Economics", "AC4105"),
    ("Advanced Tax", "AC6107"),
    ("Political Economy", "AC6111"),
    ("Major Animation Project", "MT6001"),
    ("The Case Study", "MT6003"),
    ("Animation Workplace", "MT6006"),
    ("Advanced Group Project", "CT6008"),
    ("3D Character Development", "CT6035"),
    ("Creative Awards", "AD6002"),
    ("Self Promotion", "AD6403"),
    ("Graphics Programming With Shaders", "Ct6025"),
    ("Advanced AI Algorithms", "CT6024"),
    ("Indie Game Development", "CT6018"),
    ("Strategic Marketing", "MS6101"),
    ("Capstone Bussiness Project", "BM6725"),
    ("Ai and Marketing", "MS6209"),
    ("Marketing Agency", "BM6175"),
    ("Loyalty and Communications", "MS6109"),
    ("Third Sector Marketing", "MS6210"),
    ("Teaching Practice", "SP6251"),
    ("Health Promotion for wellness and Wellbeing", "SP6368"),
    ("Media Science", "MS6126"),
]

```

Listing 42: data.py

Appendix E: Frontend

E.1 App Main

```
<script>
  import Router, { push } from "svelte-spa-router";
  import { authToken } from "../lib/stores";

  import Login from "../components/Login.svelte";
  import Dashboard from "../components/Dashboards/Dashboard.svelte";
  import Browse from "../components/Browse.svelte";
  import Navbar from "../components/Navbar.svelte";
  import Alert from "../components/Alert.svelte";

  const routes = {
    "/": Dashboard,
    "/login": Login,
    "/browse": Browse,
  };

  $: if (!$authToken) {
    push("/login");
  }
</script>

<nav>
  {#if $authToken}
    <Navbar />
  {/if}
</nav>

<Alert />
<Router {routes} />
```

Listing 43: App.svelte

E.2 API Scripts

```

import { authToken } from './stores';

const API_BASE = 'http://localhost:20550';
let token;

authToken.subscribe(value => {
  token = value;
})

async function request(path, opts = {}) {

  const url = API_BASE + path;
  const headers = opts.headers ? { ...opts.headers } : {};

  if (token) headers['Authorization'] = 'Bearer ' + token;
  headers['Content-Type'] = headers['Content-Type'] || 'application/json';

  const res = await fetch(url, { ...opts, headers });
  if (res.status === 401) {
    authToken.set(null);
    throw new Error("Unauthorized");
  }

  let data;
  const contentType = res.headers.get('content-type') || '';
  if (contentType.includes('application/json')) {
    data = await res.json();
  } else {
    data = await res.text();
  }

  if (!res.ok) {
    const err = (typeof data === 'object' && data.error) ? data.error : (data ||
res.statusText);
    throw new Error(err);
  }

  return data;
}

export async function login(username, password) {
  const res = await fetch(API_BASE + '/api/auth/login', {
    method: 'POST',
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ username, password })
  });
  if (!res.ok) {
    const err = await res.json().catch(() => ({ message: 'Login failed' }));
    throw new Error(err.message || 'bad');
  }
  const body = await res.json();
  return body.token;
}

export async function fetchBooks() {
  return request('/api/books');
}

```

Listing 44: api.js

E.3 Data Stores

```

import { derived, writable } from "svelte/store";

export const authToken = writable(localStorage.getItem('jwt_token'));
authToken.subscribe(value => {
  if (value) localStorage.setItem('jwt_token', value);
  else localStorage.removeItem('jwt_token')
});

export const alerts = writable([]);

export const filters = writable({
  startDate: new Date(Date.now() - 3600 * 24 * 365 * 1000).toISOString().slice(0, 10),
  endDate: new Date().toISOString().slice(0, 10),
});

export function addAlert(message, style) {
  const id = Math.random();

  alerts.update((prev) => [
    ...prev,
    { id, message, style }
  ]);

  setTimeout(() => {
    removeAlert(id);
  }, 5000);
}

function removeAlert(id) {
  alerts.update((prev) => prev.filter((alert) => alert.id !== id));
}

export const userRole = derived(authToken, ($auth) => {
  try {
    const payload = JSON.parse(atob($auth.split(".")[1]));
    return payload.role || null;
  } catch (err) {
    console.error("Error with the token role fetch");
    console.error(err);
    return null
  }
});

export const currentUser = derived(authToken, ($auth) => {
  try {
    const payload = JSON.parse(atob($auth.split(".")[1]));
    return payload.sub || "ERROR";
  } catch (err) {
    console.error("Error with the token username fetch");
    console.error(err);
    return null;
  }
});

```

Listing 45: stores.js

```

export const colors = [
  "#550540"

```

E.4 Component - Alert

```
<script>
  import { alerts } from "../lib/stores";
</script>

<div class="alert-container">
  {#each $alerts as alert (alert.id)}
    <div class="alert {alert.style}">
      <span>
        {alert.message}
      </span>
    </div>
  {/each}
</div>
```

Listing 46: Alert.svelte

E.5 Component - BookCard

```
<script>
  export let book;
  export let showActions = false;
  export let showDue = false;
  export let handleLoan = (_1, _2) => {};
</script>

<!-- svelte-ignore ally_click_events_have_key_events -->
<!-- svelte-ignore ally_no_static_element_interactions -->
<div class="active-card">
  <img src={book.coverURL} alt="Cover" />
  <div class="book-meta">
    <div class="book-title">{book.title}</div>
    <div class="book-author">{book.author}</div>

    {#if showActions}
      <div class="book-copies">
        {book.copiesLeft}
        ? `Copies: ${book.copiesLeft}`
        : "Out of Stock"
      </div>

      <button
        class="loanBtn"
        disabled={book.copiesLeft <= 0}
        on:click={() => handleLoan(book.id, book.title)}>Loan</button>
    >
    { :else if showDue}
      <span class="label">Return by: {book.dueDate}</span>
    {/if}
  </div>
</div>
```

Listing 47: BookCard.svelte

E.6 Component - Browse

```

<script>
  import { borrowBook, searchBooks } from "../lib/api";
  import { onMount } from "svelte";
  import BookCard from "../BookCard.svelte";
  import { addAlert } from "../lib/stores";

  const DEBOUNCE_TIME_MS = 500;

  let availableOnly = false;
  let books = [];
  let loading = true;
  let debounceTimer = null;
  let query = "";

  async function load() {
    loading = true;
    try {
      books = await searchBooks(query, availableOnly);
    } catch (err) {
      addAlert(err.message);
    } finally {
      loading = false;
    }
  }

  onMount(load);

  function onQueryChange(i) {
    query = i.target.value;
    if (debounceTimer) clearTimeout(debounceTimer);
    debounceTimer = setTimeout(() => load(), DEBOUNCE_TIME_MS);
  }

  async function handleLoanOut(bookId, bookTitle) {
    try {
      await borrowBook(bookId);
      addAlert(`Book borrowed: ${bookTitle}`, "success");
      await load();
    } catch (err) {
      addAlert(err.message, "error");
    }
  }
}
</script>

<div>
  <h2>Browse Books</h2>

  <div class="controls">
    <input
      placeholder="Search by title or author..."
      value={query}
      on:input={onQueryChange}
    />
    <label>
      <input
        type="checkbox"
        bind:checked={availableOnly}
        on:change={handleLoanOut}
      /> Available only</label>
    </div>
  </div>

```

E.7 Component - Login

```
<script>
  import { push } from "svelte-spa-router";
  import { login } from "../lib/api";
  import { authToken } from "../lib/stores";

  let username = "user";
  let password = "password";
  let error = "";
  let loading = false;

  async function onSubmit(event) {
    event.preventDefault();
    error = "";
    loading = true;
    try {
      const token = await login(username, password);

      authToken.set(token);

      push("/");
    } catch (err) {
      error = err.message || "Failed to login. Please check creds";
    } finally {
      loading = false;
    }
  }
</script>

<div class="panel login-form">
  <h2>Login</h2>
  <form on:submit={onSubmit}>
    <label> Username <input bind:value={username} /> </label>

    <label> Password <input bind:value={password} type="password" /></label>

    <div style="margin-top:10px;">
      <button type="submit" disabled={loading}
        >{loading ? "Logging in..." : "Login"}</button>
    >
  </div>

  {#if error}
    <p style="color:red;">{error}</p>
  {/if}
</form>
</div>
```

Listing 49: Login.svelte

E.8 Component - Navbar

```
<script>
  import { link } from "svelte-spa-router";
  import { authToken, currentUser, userRole } from "../lib/stores";

  function logout() {
    authToken.set(null);
  }
</script>

<nav class="navbar">
  <div class="container nav-container">
    <a use:link href="/" class="nav-brand">DBA2<span>Library</span></a>
    <div class="nav-links">
      {#if $userRole === "STUDENT"}
        <a use:link href="/browse" class="nav-item">Browse</a>
      {/if}

      <div class="ctaMenu">
        <button on:click={logout} class="nav-logout">Logout</button>
        {#if $currentUser}
          <div class="label">
            Logged in as: <strong>{$currentUser}</strong>
          </div>
        {/if}
      </div>
    </div>
  </div>
</nav>
```

Listing 50: Navbar.svelte

E.9 Component - Dashboard

```
<script>
  import { userRole } from "../../lib/stores";
  import StaffDashboard from "../StaffDashboard.svelte";
  import StudentDashboard from "../StudentDashboard.svelte";
</script>

{#if $userRole === "STUDENT"}
  <StudentDashboard />
{:else if ["ADMIN", "DECISION_MAKER"].includes($userRole)}
  <StaffDashboard />
{:else}
  <p>Loading...</p>
{/if}
```

Listing 51: Dashboard.svelte

E.10 Component - Staff Dashboard

```

<script>
  import { startEtl } from "../../lib/api";
  import { userRole, addAlert, filters } from "../../lib/stores";
  import BackupPanel from "../BackupPanel.svelte";
  import ChartCard from "../ChartCard.svelte";
  import CourseEngagementChart from "../Charts/CourseEngagementChart.svelte";
  import DurationChart from "../Charts/DurationChart.svelte";
  import FineRevChart from "../Charts/FineRevChart.svelte";
  import LoanChart from "../Charts/LoanChart.svelte";
  import OverdueChart from "../Charts/OverdueChart.svelte";
  import PeakDaysChart from "../Charts/PeakDaysChart.svelte";
  import PopularChart from "../Charts/PopularChart.svelte";
  import YearEngagement from "../Charts/YearEngagement.svelte";

  let startDate = "";
  let endDate = "";

  filters.subscribe((evt) => {
    startDate = evt.startDate;
    endDate = evt.endDate;
  });

  async function handleFilter() {
    addAlert("Changed filters", "success");
    filters.set({ startDate, endDate });
  }

  async function handleEtl() {
    try {
      addAlert(`ETL Process started..`, `success`);
      await startEtl();
      addAlert(`Finished ETL`, `success`);

      // This isnt ideal. Should probably do a key block or load data in this
parent
      window.location.reload();
    } catch (err) {
      addAlert(err.message);
    }
  }
</script>

<div>
  <label> From: <input type="date" bind:value={startDate} /></label>
  <label> To: <input type="date" bind:value={endDate} /></label>

  <button on:click={() => handleFilter()}>Apply</button>
</div>

<div class="dashboard-grid">
  <div style="grid-column: span 3">
    <ChartCard title="Loans Trend Over Time">
      <LoanChart />
    </ChartCard>
  </div>

  {#if $userRole == "ADMIN"}
    <ChartCard title="Admining 52r Staff Dashboard.svelte">
      <button on:click={() => handleEtl()}>Load ETL</button>
      <BackupPanel />
    </ChartCard>
  {/if}

```

E.11 Component - Student Dashboard

```

<script>
  import { push } from "svelte-spa-router";
  import { fetchLoansForUser, payFine, returnBook } from "../lib/api";
  import { onMount } from "svelte";
  import BookCard from "../BookCard.svelte";
  import LoanPanel from "../loanPanel.svelte";
  import { addAlert } from "../lib/stores";

  let loans = [];
  let loading = true;

  async function handlePay(loanId, fineAmount) {
    if (!confirm(`Are you sure?\nPayment Due: £${fineAmount}`)) return;
    try {
      await payFine(loanId);
      addAlert(`Payment of £${fineAmount} complete!`, "success");
      await load();
    } catch (err) {
      addAlert(err.message);
    }
  }

  async function loanReturn(loanId) {
    try {
      await returnBook(loanId);
      addAlert(`Book returned`, "success");
      await load();
    } catch (err) {
      addAlert(err.message);
    }
  }

  async function load() {
    try {
      loans = await fetchLoansForUser();
    } catch (err) {
      addAlert(err.message);
      if (err.message === "Unauthorized") {
        push("/login");
      }
    } finally {
      loading = false;
    }
  }

  onMount(() => {
    load();
  });

  $: currentLoans = loans.filter((loan) => !loan.returnDate);
  $: finesList = loans.filter((loan) => loan.status === "OVERDUE");
</script>

```

```

<section>

```

```

  <h3>Reading List</h3>

```

```

  {#if currentLoans.length === 0}

```

```

    <p>No books currently on loan!</p>

```

```

  {:else}

```

```

    <div class="grid">

```

```

      {#each currentLoans as loan, i}

```

```

        <BookCard

```

```

          book={{ ...loan.book, dueDate: loan.dueDate }}

```

```

          showDue={true}

```

E.12 Component - LoanPanel

```
<script>
  export let loans = [];
  export let onReturn;

  let startDate = "";
  let endDate = "";

  $: loansHistory = loans
    .filter((loan) => {
      if (!startDate && !endDate) return true;

      const loanDate = new Date(loan.loanDate);
      const start = startDate
        ? new Date(startDate)
        : new Date("2010-01-01");
      const end = endDate ? new Date(endDate) : new Date(Date.now());

      return loanDate >= start && loanDate <= end;
    })
    .sort((l1, l2) => {
      return (
        new Date(l2.loanDate).getTime() -
        new Date(l1.loanDate).getTime()
      );
    });

  $: totalPaid = loansHistory
    .filter((loan) => loan.status === "PAID")
    .reduce((sum, loan) => sum + (loan.fine || 0), 0);

  function genTags(loan) {
    let tagList = [];

    if (loan.returnDate) {
      tagList.push({ text: "Returned", style: "returned" });

      if (new Date(loan.returnDate) > new Date(loan.dueDate)) {
        tagList.push({ text: "Late", style: "late" });
      }
    } else if (loan.status === "OVERDUE") {
      tagList.push({ text: "Overdue", style: "late" });
    } else {
      tagList.push({ text: "Borrowed", style: "active" });
    }

    return tagList;
  }
</script>
```

E.13 Component - Backup Panel

```

<script>
  import { onMount } from "svelte";
  import { createBackup, listBackups, restoreBackup } from "../lib/api";
  import { addAlert } from "../lib/stores";

  let backups = [];
  let loading = true;
  let creating = false;
  let restoringFile = null;

  async function load() {
    try {
      backups = await listBackups();
    } catch (err) {
      addAlert(err.message);
    } finally {
      loading = false;
    }
  }

  async function handleCreate() {
    creating = true;
    try {
      const res = await createBackup();
      addAlert(`Backup created: ${res.filename}`, "success");
      await load();
    } catch (err) {
      addAlert(err.message);
    } finally {
      creating = false;
    }
  }

  async function handleRestore(filename) {
    if (
      !confirm(
        `Restore from {filename}? This will overwrite the current data`,
      )
    )
      return;
    restoringFile = filename;
    try {
      handleCreate();

      await restoreBackup(filename);
      addAlert("Restored from previous backup");
    } catch (err) {
      addAlert(err.message);
    } finally {
      restoringFile = null;
    }
  }

  onMount(load);
</script>

<button class="create-btn" on:click={handleCreate} disabled={creating}
  >{creating ? "In Progress" : "Create Backup"}</button>
>

```

Listing 55: Backup Panel Svelte

```

{#if loading}
  <p>Loading...</p>
</if>

```

E.14 Component - Chart Card

```
<script>
  export let title = "";
</script>

<div class="card">
  {#if title}
    <h3>{title}</h3>
  {/if}
  <div style="height: 280px">
    <slot />
  </div>
</div>

<style>
</style>
```

Listing 56: ChartCard.svelte

E.15 Component - Course Engagement Chart

```
<script>
  import {
    ArcElement,
    Chart,
    DoughnutController,
    Legend,
    Tooltip,
  } from "chart.js";
  import { fetchCourseEngagment } from "../../lib/api";
  import { colors, filters } from "../../lib/stores";
  import { Doughnut } from "svelte-chartjs";

  Chart.register(ArcElement, DoughnutController, Tooltip, Legend);

  let data = { labels: [], datasets: [] };
  let loading = true;

  async function load(evt) {
    loading = true;
    const rows = await fetchCourseEngagment(evt.startDate, evt.endDate);
    data = {
      labels: rows.map((row) => row.courseName),
      datasets: [
        {
          data: rows.map((row) => row.loanCount),
          backgroundColor: colors,
        },
      ],
    };
    loading = false;
  }
  $: load($filters);

  const options = {
    responsive: true,
    maintainAspectRatio: false,
    plugins: {
      legened: { position: "right" },
    },
  };
</script>

{#if loading}
  <p>Loading...</p>
{:else}
  <Doughnut {data} {options} />
{/if}
```

Listing 57: CourseEngagementChart.svelte

E.16 Component - Duration Chart

```

<script>
  import {
    BarController,
    BarElement,
    CategoryScale,
    Chart,
    Legend,
    LinearScale,
    Title,
    Tooltip,
  } from "chart.js";
  import { onMount } from "svelte";
  import { fetchLoanDurationByCourse } from "../../lib/api";
  import { Bar } from "svelte-chartjs";
  import { colors, filters } from "../../lib/stores";

  Chart.register(
    BarElement,
    BarController,
    CategoryScale,
    LinearScale,
    Title,
    Tooltip,
    Legend,
  );

  let data = { labels: [], datasets: [] };
  let loading = true;

  async function load(evt) {
    loading = true;
    const rows = await fetchLoanDurationByCourse(
      evt.startDate,
      evt.endDate,
    );
    data = {
      labels: rows.map((row) => row.courseName),
      datasets: [
        {
          label: "Average Loan duration",
          data: rows.map((row) => row.avg),
          backgroundColor: colors,
        },
      ],
    };
    loading = false;
  }
  $: load($filters);

  const options = {
    responsive: true,
    maintainAspectRatio: false,
    plugins: {
      legend: { position: "top" },
    },
    scales: { y: { beginAtZero: true } },
  };
</script>

```

Listing 58: DurationChart.svelte

lxxiii

```

{#if loading}
  <p>loading...</p>
{:else}

```

E.17 Component - Fine Revenue Chart

```

<script>
  import {
    BarElement,
    CategoryScale,
    Chart,
    Legend,
    LinearScale,
    Tooltip,
  } from "chart.js";
  import BookCard from "../BookCard.svelte";
  import { fetchFineRevenue } from "../../lib/api";
  import { color } from "chart.js/helpers";
  import { colors, filters } from "../../lib/stores";
  import { Bar } from "svelte-chartjs";

  Chart.register(BarElement, CategoryScale, Legend, LinearScale, Tooltip);

  let data = { labels: [], datasets: [] };
  let loading = true;

  async function load(evt) {
    loading = true;
    const rows = await fetchFineRevenue(evt.startDate, evt.endDate);

    const months = [
      ...new Set(rows.map((row) => `${row.monthName} ${row.year}`)),
    ];
    const courses = [...new Set(rows.map((row) => row.courseName))];

    const datasets = courses.map((course, i) => ({
      label: course,
      data: months.map((month) => {
        const row = rows.find(
          (row) =>
            `${row.monthName} ${row.year}` === month &&
            row.courseName === course,
        );
        return row ? row.totalFines : 0;
      }),
      backgroundColor: colors,
    }));

    data = { labels: months, datasets };
    loading = false;
  }
  $: load($filters);

  const options = {
    responsive: true,
    maintainAspectRatio: false,
    scales: { x: { stacked: true }, y: { stacked: true } },
    plugins: { Legend: { display: false, position: "flase" } },
  };
</script>

{#if loading}
  <p>Loading...</p>
{:else}
  <Bar {data} {options} />
{/if}

```

Listing 59: FineRevChart.svelte

E.18 Component - Loan Chart

```

<script>
  import {
    CategoryScale,
    Chart,
    Filler,
    Legend,
    LinearScale,
    LineController,
    LineElement,
    PointElement,
    Title,
    Tooltip,
  } from "chart.js";
  import { onMount } from "svelte";
  import { Line } from "svelte-chartjs";
  import { fetchLoanTrend } from "../../lib/api";
  import { filters } from "../../lib/stores";

  Chart.register(
    LineElement,
    PointElement,
    LineController,
    CategoryScale,
    LinearScale,
    Title,
    Tooltip,
    Legend,
    Filler,
  );

  let data = { labels: [], datasets: [] };
  let loading = true;

  async function load(evt) {
    loading = true;
    const rows = await fetchLoanTrend(evt.startDate, evt.endDate);
    data = {
      labels: rows.map((row) => `${row.monthName} ${row.year}`),
      datasets: [
        {
          label: "Total Loans",
          data: rows.map((row) => row.totalLoans),
          borderColor: "#6366f1",
          tension: 0.3,
          fill: true,
        },
      ],
    };
    loading = false;
  }
  $: load($filters);

  const options = {
    responsive: true,
    maintainAspectRatio: false,
    plugins: {
      legend: { position: "top" },
    },
    scales: {
      y: { beginAtZero: true },
    },
  };

```

E.19 Component - Overdue Chart

```

<script>
  import {
   BarController,
    BarElement,
    CategoryScale,
    Chart,
    Legend,
    LinearScale,
    Title,
    Tooltip,
  } from "chart.js";
  import { onMount } from "svelte";
  import { fetchOverdueSummary } from "../../lib/api";
  import { Bar } from "svelte-chartjs";

  Chart.register(
    BarElement,
    BarController,
    CategoryScale,
    LinearScale,
    Title,
    Tooltip,
    Legend,
  );

  let data = { labels: [], datasets: [] };
  let loading = true;

  onMount(async () => {
    const rows = await fetchOverdueSummary();
    data = {
      labels: rows.map((row) => row.courseName),
      datasets: [
        {
          label: "Overdue Loans",
          data: rows.map((row) => row.overdueCount),
          backgroundColor: "#ef4444",
        },
        {
          label: "Total Owned: £",
          data: rows.map((row) => row.totalFines),
          backgroundColor: "#4444ff",
        },
      ],
    };
    loading = false;
  });

  const options = {
    responsive: true,
    maintainAspectRatio: false,
    plugins: {
      legend: { position: "top" },
    },
    scales: { y: { beginAtZero: true } },
  };
</script>

```

Listing 61: OverdueChart.svelte

lxxix

```

{#if loading}
  <p>Loading...</p>
{:else}

```

E.20 Component - Popular Chart

```

<script>
  import {
   BarController,
    BarElement,
    CategoryScale,
    Chart,
    Legend,
    LinearScale,
    Title,
    Tooltip,
  } from "chart.js";
  import { onMount } from "svelte";
  import { fetchPopular } from "../../lib/api";
  import BookCard from "../BookCard.svelte";
  import { Bar } from "svelte-chartjs";
  import { filters } from "../../lib/stores";

  Chart.register(
    BarElement,
    BarController,
    CategoryScale,
    LinearScale,
    Title,
    Tooltip,
    Legend,
  );

  let data = { labels: [], datasets: [] };
  let loading = true;

  async function load(evt) {
    loading = true;
    const rows = await fetchPopular(evt.startDate, evt.endDate);
    data = {
      labels: rows.map((row) => row.title),
      datasets: [
        {
          label: "Times Borrowed",
          data: rows.map((row) => row.loanCount),
          backgroundColor: "#981da6",
        },
      ],
    };
    loading = false;
  }
  $: load($filters);

  const options = {
    indexAxis: "y",
    responsive: true,
    maintainAspectRatio: false,
    plugins: {
      legend: { position: "top" },
    },
    scales: { x: { beginAtZero: true } },
  };
</script>

```

Listing 62: PopularChart.svelte

```

{#if loading}
  <p>Loading...</p>
{:else}
  <Bar {data} {options} />
{/if}

```

E.21 Component - Year Engagement Chart

```
<script>
  import {
    ArcElement,
    Chart,
    DoughnutController,
    Legend,
    Tooltip,
  } from "chart.js";
  import { onMount } from "svelte";
  import { fetchYearEngagment } from "../lib/api";
  import { Doughnut } from "svelte-chartjs";
  import { colors, filters } from "../lib/stores";

  Chart.register(ArcElement, DoughnutController, Tooltip, Legend);

  let data = { labels: [], datasets: [] };
  let loading = true;

  let years = { 1: "1st Year", 2: "2nd Year", 3: "3rd Year (last)" };

  async function load(evt) {
    loading = true;
    const rows = await fetchYearEngagment(evt.startDate, evt.endDate);
    data = {
      labels: rows.map(
        (row) => years[row.studyYear] ?? `Year ${row.studyYear}`,
      ),
      datasets: [
        {
          data: rows.map((row) => row.loanCount),
          backgroundColor: colors,
        },
      ],
    };
    loading = false;
  }
  $: load($filters);

  const options = {
    responsive: true,
    maintainAspectRatio: false,
    plugins: {
      legened: { position: "right" },
    },
  };
</script>

{#if loading}
  <p>Loading...</p>
{:else}
  <Doughnut {data} {options} />
{/if}
```

Listing 63: YearEngagement.svelte

Appendix F: Miscellaneous

F.1 Java Application Yaml

```
spring:
  application:
    name: backend
  datasource:
    url: jdbc:postgresql://localhost:5432/libdb
    username: libuser
    password: libpass
  jpa:
    hibernate:
      ddl-auto: validate
      show-sql: false
  flyway:
    enabled: true

server:
  port: 20550

jwt:
  secret: "123456789123456789123456789123456789"
  expiration-ms: 86400000

cors:
  allowed-origins: http://localhost:5173

logging:
  level:
    org:
      hibernate:
        SQL: DEBUG
      orm:
        jdbc:
          bind: TRACE
```

Listing 64: application.yaml

F.2 Java pom.xml

```

<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/
XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/
maven-v4_0_0.xsd">
    <modelVersion>4.0.0</modelVersion>

```

```

<parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.5.7</version>
    <relativePath/>
</parent>

```

```

    <groupId>github.peaterpita</groupId>
    <artifactId>backend</artifactId>
    <packaging>jar</packaging>
    <version>1.0-SNAPSHOT</version>
    <name>backend</name>
    <url>http://maven.apache.org</url>
    <dependencies>
        <dependency>
            <groupId>junit</groupId>
            <artifactId>junit</artifactId>
            <version>3.8.1</version>
            <scope>test</scope>
        </dependency>

        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>

        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>

        <dependency>
            <groupId>javax.persistence</groupId>
            <artifactId>persistence-api</artifactId>
            <version>1.0.2</version>
        </dependency>

        <dependency>
            <groupId>org.postgresql</groupId>
            <artifactId>postgresql</artifactId>
            <version>42.6.0</version>
        </dependency>

        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-redis</artifactId>
        </dependency>

        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-security</artifactId>
        </dependency>

```

Listing 65: pom.xml

lxxxv

F.3 Nix Flake

```

{
  description = "Nix-Flake based Java Dev Shell";
  inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-unstable";

  outputs =
    inputs:
    let
      javaVersion = 21;
      system = "x86_64-linux";

      run-appScript = pkgs.writeShellScriptBin "run-app" ''
        trap "kill 0" SIGINT SIGTERM EXIT

        echo "-----"
        echo "STARTING LIBRARY SYSTEM"
        echo "-----"

        if ! sudo docker ps | grep -q "postgres"; then
          echo "Starting Docker containers..."
          sudo docker compose up -d > /dev/null 2>&1
          sleep 3
        fi

        echo "Starting Backend..."
        (cd backend && mvn spring-boot:run > ../backend.log 2>&1) &

        echo "Starting Frontend..."
        (cd frontend && npm run dev -- > ../frontend.log 2>&1) &

        echo "-----"
        echo "  Application Running!"
        echo "  Frontend: http://localhost:5173"
        echo "  Backend:  http://localhost:20550"
        echo "-----"

        wait
      '';

      pkgs = import inputs.nixpkgs {
        inherit system;
        overlays = [ overlay ];
      };

      overlay =
        final: prev:
        let
          jdk = prev."jdk${toString javaVersion}";
        in
        {
          inherit jdk;
          maven = prev.maven.override { jdk_headless = jdk; };
        };

    in
    {

      devShells.${system}.default = pkgs.mkShell {
        packages = with pkgs; [
          gcc
          nodejs
          jdk
          maven
          lib
        ];
      };
    }
  };
}

```

F.4 Docker Compose

```
services:
  postgres:
    image: postgres
    container_name: lib_postgres
    environment:
      POSTGRES_USER: libuser
      POSTGRES_PASSWORD: libpass
      POSTGRES_DB: libdb
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql

  redis:
    image: redis:alpine
    container_name: lib_redis
    ports:
      - "6379:6379"

  pgadmin:
    image: adminer
    restart: always
    ports:
      - "8080:8080"
    depends_on:
      - postgres

volumes:
  pgdata:
```

Listing 67: docker-compose.yml